Simulação de marcha de amputado transfemoral com prótese passiva: dinâmica direta OpenSim integrada no MATLAB e controle muscular por aprendizado por reforço

Relatório técnico do projeto `RL_OSim`

5 de novembro de 2026

Resumo

Este relatório documenta o código desenvolvido para simular, por dinâmica direta, a marcha de um amputado transfemoral direito com o modelo OpenSim `OS4_gait14dof15musc_2Act_RTFP_DynAct.osim`. Joelho e tornozelo protéticos são *passivos* (mola e amortecedor), e os 15 músculos do modelo são controlados por um agente *Soft Actor-Critic* (SAC) que aprende correções sobre um controlador-base de rastreamento. Toda a integração das equações diferenciais é feita no MATLAB (`ode23`); o OpenSim apenas avalia $\dot{\mathbf{y}} = f(t, \mathbf{y}, \mathbf{e})$. A recompensa segue o estilo DeepMimic: imita a cinemática de marcha saudável, busca 1,2 m/s e penaliza esforço e variações bruscas de excitação. Depois de 150 blocos de 40 episódios, o melhor agente anda 20 s sem cair a partir de qualquer fase inicial testada, a 0,94 m/s e com assimetria de apoio de 11 %. Ele atende 3 dos 5 critérios de plausibilidade fisiológica adotados. O relatório descreve a arquitetura, a função de recompensa, as redes neurais, o procedimento de treino, os resultados e cada arquivo do código.

Sumário

1 Objetivo e requisitos **3**

2 Arquitetura geral **3**

3 Modelo musculoesquelético e prótese passiva **4**

4 Dinâmica direta integrada no MATLAB **4**

5 Ambiente de aprendizado por reforço **5**
5.1 Observação (63 elementos) 5
5.2 Ação e composição das excitações 5
5.3 Controlador-base 5
5.4 Referência de marcha saudável 6
5.5 Início, término e duração dos episódios 6

6 Função de recompensa **6**

7 Agente SAC e estrutura das redes neurais **7**
7.1 Algoritmo 7
7.2 Arquitetura 7
7.3 Hiperparâmetros 9

8 Procedimento de treino **9**
8.1 Histórico das execuções 9

9 Avaliação e seleção do agente **10**
9.1 Métricas 10
9.2 Comparação robusta 10

10 Resultados do melhor agente **11**

11 Comparação com a marcha saudável por ciclo **12**
11.1 Dados saudáveis e procedimento 12
11.2 Resultados 14

12 Visualização **15**

13 Desempenho computacional **15**

14 Problemas encontrados e soluções **15**

15 Limitações e próximos passos **16**

16 Guia de uso **16**

A Referência dos arquivos **17**

B Código-fonte **17**
B.1 `gaitConfig.m` 17
B.2 `healthyGaitReference.m` 19
B.3 `ProstheticGaitEnv.m` 20
B.4 `train_sac.m` 26
B.5 `evaluate_policy.m` 27
B.6 `compare_agents.m` 29
B.7 `gait_cycle_comparison.m` 30
B.8 `make_media.m` 32
B.9 `render_opensim.m` 35
B.10 `capture_window.ps1` 37
B.11 `bench_gpu.m` 37
B.12 `test_deriv.m` 37
B.13 `test_baseline.m` 38
B.14 `sweep.m` 38

1 Objetivo e requisitos

O objetivo é emular, no modelo de um amputado transfemoral, uma marcha com prótese o mais próxima possível da marcha humana saudável, controlando os músculos do modelo por aprendizado por reforço até que o movimento seja suave o bastante para parecer fisiologicamente possível. Os requisitos fixados ao longo do projeto foram:

1. **Modelo:** `OS4_gait14dof15musc_2Act_RTFP_DynAct.osim` (amputação transfemoral direita).
2. **Músculos controlados** (15): `abd_r`, `add_r`, `glut_max_r`, `iliopsoas_r` (coto) e `abd_l`, `add_l`, `hamstrings_l`, `bifemsh_l`, `glut_max_l`, `iliopsoas_l`, `rect_fem_l`, `vasti_l`, `gastroc_l`, `soleus_l`, `tib_ant_l` (perna intacta).
3. **Integração da ODE no MATLAB**, não no OpenSim (o `Manager` do OpenSim não é usado).
4. **Prótese passiva:** joelho e tornozelo sem nenhum controle ativo, como em próteses passivas reais.

2 Arquitetura geral

A Figura 1 mostra o laço de controle. A cada passo de controle de 10 ms, o agente lê uma observação de 63 elementos e devolve 15 resíduos de excitação. Esses resíduos passam por um filtro passa-baixa e são somados às excitações do controlador-base. As excitações resultantes ficam constantes durante o passo (*zero-order hold*) enquanto o `ode23` do MATLAB integra o modelo OpenSim.

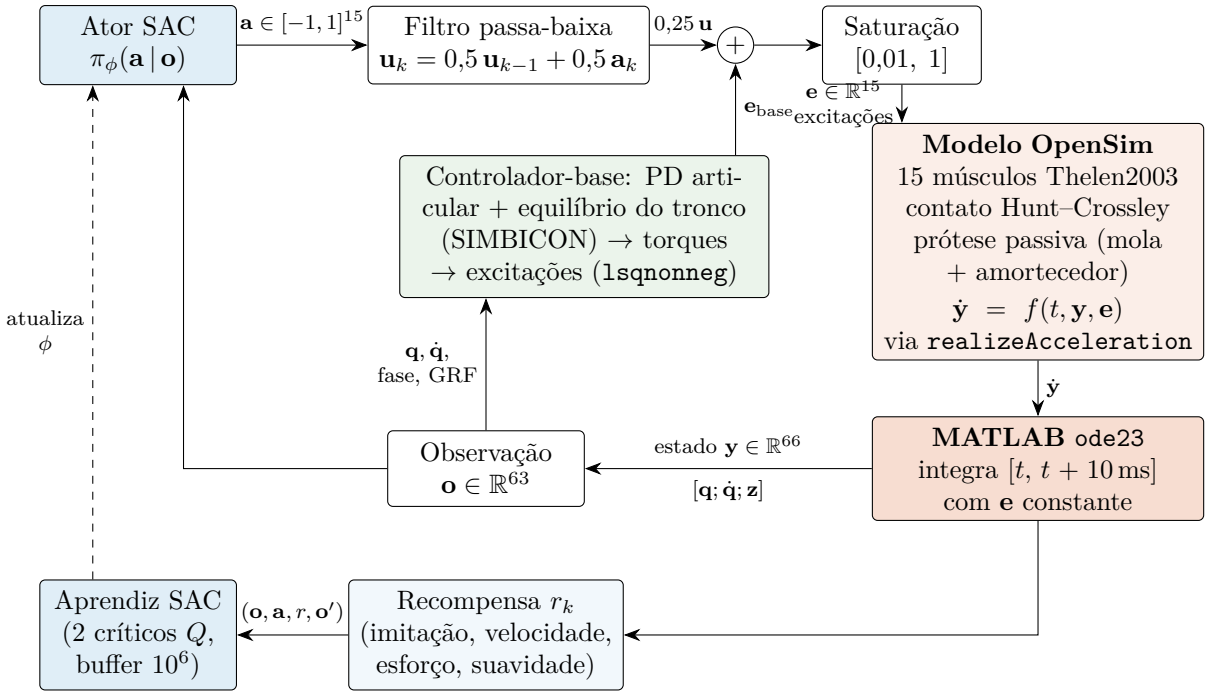


Figura 1: Laço de controle. O OpenSim só avalia as derivadas; a integração é feita pelo `ode23` do MATLAB. O agente produz resíduos sobre o controlador-base, e a prótese não recebe nenhum comando.

Tabela 1: Divisão de responsabilidades.

Parte	Onde
Equações do modelo, $\dot{\mathbf{y}} = f(t, \mathbf{y}, \mathbf{e})$	OpenSim 4.4 (API Java): `setY` → `realizeAcceleration` → `getYDot`
Integração da ODE	MATLAB, `ode23` (RelTol 10^{-3}, AbsTol 10^{-4})
Controle (constante por 10 ms)	MATLAB: controlador-base + resíduo do agente
Aprendizado por reforço	MATLAB Reinforcement Learning Toolbox, agente SAC, 4 workers em paralelo

3 Modelo musculoesquelético e prótese passiva

O modelo tem pelve, tronco, perna esquerda intacta e o lado direito amputado: coto femoral (`transfemur`), encaixe osseointegrado (`osseo_pylon`), pilão tibial (`tibia_pylon`) e pé protético (`foot_r`). Os 15 músculos são do tipo Thelen2003. O contato com o solo usa esferas Hunt–Crossley no calcanhar e na ponta de cada pé (`foot_r`, `foot_l`).

Estágio planar. Para o primeiro estágio, a marcha é restrita ao plano sagital, com abdução e adução do quadril livres. Ficam travadas as coordenadas `pelvis_list`, `pelvis_rotation`, `pelvis_tz` e `hip_rotation_r` (`gaitConfig('planar',true)`, o padrão).

Prótese passiva. Os atuadores ativos do modelo, `knee_actuator` e `ankle_actuator`, são desligados (`set_appliesForce(false)`). Em cada junta protética entra um `SpringGeneralizedForce` que aplica

$$\tau_{\text{prot}} = -k\,(q - q_0) - b\,\dot{q}, \tag{1}$$

com os valores da Tabela 2. Os batentes `KneeLimit_r` e `AnkleLimit_r` do modelo continuam ativos. Assim, joelho e tornozelo direitos só se movem pela dinâmica do corpo e pelas forças dos músculos do coto sobre o fêmur residual, como numa prótese passiva real.

Tabela 2: Parâmetros da prótese passiva (`c.prosthesis` em `gaitConfig.m`).

Junta	k (N m/rad)	b (N m s/rad)	q_0	Interpretação
Joelho (`knee_angle_r`)	5	3	0	joelho mecânico com atrito hidráulico simples e leve mola de extensão
Tornozelo (`ankle_angle_r`)	350	2	0	pé de armazenamento de energia (ESAR), ≈6 N m/grau

Assistência na pelve. O código cria também atuadores de coordenada na pelve (`assist_*`), uma "mão de Deus" para currículo. Na execução final eles ficam desligados (`assistStart=0`), e toda avaliação é feita sem assistência.

4 Dinâmica direta integrada no MATLAB

O estado do Simbody tem 66 variáveis:

$$\mathbf{y} = \left[\, \mathbf{q} \in \mathbb{R}^{17};\ \dot{\mathbf{q}} \in \mathbb{R}^{17};\ \mathbf{z} \in \mathbb{R}^{32} \,\right], \tag{2}$$

em que $\mathbf{z}$ reúne 15 ativações musculares, 15 comprimentos de fibra e 2 ativações dos atuadores protéticos desligados, que ficam inertes. A função `dynamics` do ambiente é o lado direito da ODE:

```
function yd = dynamics(this, s, t, y)
    s.setTime(t);
    s.setY(org.opensim.modeling.Vector.createFromMat(y));
    this.model.realizeAcceleration(s);
    yd = s.getYDot().getAsMat();
end
```

Em cada passo de controle k, as excitações $\mathbf{e}_k$ são gravadas em funções `Constant` de um `PrescribedController`, e o MATLAB integra o intervalo:

$$\mathbf{y}(t_k + \Delta t) = \mathbf{y}(t_k) + \int_{t_k}^{t_k+\Delta t} f(\tau, \mathbf{y}(\tau), \mathbf{e}_k)\, d\tau, \qquad \Delta t = 10 \text{ ms}. \tag{3}$$

O integrador é o `ode23` (Bogacki–Shampine), com RelTol $= 10^{-3}$ e AbsTol $= 10^{-4}$. Ele foi o mais rápido dentre `ode45`, `ode23`, `ode15s`, `ode23t` e `ode23tb` no teste `test_deriv.m`. Se a integração diverge ou o estado fica inválido, o episódio é encerrado como queda.

5 Ambiente de aprendizado por reforço

O ambiente é a classe `ProstheticGaitEnv`, derivada de `rl.env.MATLABEnvironment`.

5.1 Observação (63 elementos)

Tabela 3: Composição da observação $\mathbf{o}$ (função `observe`).

Bloco	Conteúdo	Tamanho
$\mathbf{q}$	`pelvis_tilt`, `pelvis_ty`, flexão e adução dos dois quadris, joelhos e tornozelos (sem `pelvis_tx`)	10
$0{,}1\,\dot{\mathbf{q}}$	velocidades das mesmas coordenadas, incluindo `pelvis_tx`	11
$\mathbf{a}_{\text{mus}}$	ativações dos 15 músculos	15
GRF	força vertical do solo em cada pé, dividida pelo peso corporal	2
fase	$\sin(2\pi\varphi)$, $\cos(2\pi\varphi)$ da fase do ciclo de referência	2
$\hat{\mathbf{q}}$	ângulos de referência das 8 juntas rastreadas no próximo passo	8
$\mathbf{u}_{k-1}$	ação filtrada anterior	15
Total		**63**

5.2 Ação e composição das excitações

A ação é $\mathbf{a} \in [-1, 1]^{15}$, um resíduo por músculo. As excitações aplicadas são

$$\mathbf{u}_k = 0{,}5\,\mathbf{u}_{k-1} + 0{,}5\,\mathbf{a}_k, \tag{4}$$

$$\mathbf{e}_k = \text{clip}\big(\mathbf{e}_{\text{base},k} + 0{,}25\,\mathbf{u}_k,\ 0{,}01,\ 1\big). \tag{5}$$

5.3 Controlador-base

O controlador-base (função `baseline`) gera uma marcha aproximada que o agente corrige. Para as seis juntas acionadas por músculos (flexão e adução dos dois quadris, joelho e tornozelo esquerdos), ele calcula torques PD em torno da referência saudável no instante seguinte:

$$\tau_j = K_{p,j}\,(\hat{q}_j - q_j) + K_{d,j}\,(\dot{\hat{q}}_j - \dot{q}_j). \tag{6}$$

O tronco é equilibrado pelo quadril de apoio, como no SIMBICON. Um torque proporcional à inclinação da pelve é distribuído entre os quadris conforme a fração de carga de cada pé:

$$\tau^{R,L}_{\text{quadril}} \mathrel{+}= w^{R,L}\,(K_{p,T}\,\theta + K_{d,T}\,\dot{\theta}), \qquad w^R = \frac{F_R}{F_R + F_L},\ w^L = 1 - w^R. \tag{7}$$

Os torques viram excitações por um problema de mínimos quadrados não negativos, semelhante a uma otimização estática, em que $\mathbf{R}$ é a matriz de braços de momento, recalculada a cada 3 passos:

$$\mathbf{e}_{\text{base}} = \min\Big(1,\ 0{,}02 + \arg\min_{\mathbf{e}\geq 0}\ \big\|(\mathbf{R}\circ\mathbf{F}_{\max})\,\mathbf{e} - \boldsymbol{\tau}\big\|^2 + \lambda\,\big\|\operatorname{diag}(\mathbf{F}_{\max})\,\mathbf{e}\big\|^2\Big), \qquad \lambda = 10^{-4}. \tag{8}$$

Nenhum torque é pedido ao joelho ou ao tornozelo protéticos.

Tabela 4: Ganhos do controlador-base (`gaitConfig.m`).

	quadril D	adução D	quadril E	adução E	joelho E	tornozelo E	tronco
K_p (N m/rad)	250	150	250	150	200	150	1500
K_d (N m s/rad)	15	10	15	10	12	8	100

5.4 Referência de marcha saudável

`healthyGaitReference.m` gera curvas periódicas de quadril, joelho, tornozelo e adução do quadril a partir de pontos-chave da literatura (Winter 2009; Perry 1992), para 1,2 m/s e ciclo de 1,1 s. Os pontos são interpolados por `pchip` periódico e suavizados mantendo 8 harmônicos de Fourier (1000 amostras por ciclo). A perna direita usa a mesma curva defasada de meio ciclo.

5.5 Início, término e duração dos episódios

- **Inicialização pelo estado de referência (RSI):** cada episódio começa numa fase aleatória do ciclo, com pose e velocidades da referência, pelve a 1,2 m/s e o pé mais baixo tocando o solo. Se `equilibrateMuscles` falhar, outra fase é sorteada (até 6 tentativas).
- **Queda:** altura da pelve $< 0{,}70$ m ou |inclinação| $> 0{,}6$ rad. A queda encerra o episódio com penalidade de 2.
- **Duração:** 600 passos (6 s) a partir do bloco 94. Antes eram 300 passos (3 s).

6 Função de recompensa

A recompensa (função `reward`) segue o estilo DeepMimic, em forma multiplicativa:

$$\boxed{r_k = \underbrace{\big(w_p\, r_{\text{pose}} + w_v\, r_{\text{vel}} + w_s\, r_{\text{speed}} + w_u\, r_{\text{up}} + w_{\text{alive}}\big)}_{\text{tarefa}} \cdot \exp\big(-P_k\big)\ -\ 2\cdot\mathbf{1}[\text{queda}]} \tag{9}$$

com os termos de tarefa

$$r_{\text{pose}} = \exp\Big(-k_p \sum_j \omega_j\,(q_j - \hat{q}_j)^2\Big), \qquad r_{\text{vel}} = \exp\Big(-k_v \sum_j \omega_j\,(\dot{q}_j - \dot{\hat{q}}_j)^2\Big), \tag{10}$$

$$r_{\text{speed}} = \exp\big(-k_s\,(v_x - 1{,}2)^2\big), \qquad r_{\text{up}} = \exp\big(-k_\theta\,\theta^2 - k_h\,(y_p - y_{p,0})^2\big), \tag{11}$$

e a soma das penalidades

$$P_k = w_e\,\overline{\mathbf{e}_k^{\,3}} + w_\Delta\,\overline{(\mathbf{e}_k - \mathbf{e}_{k-1})^2} + w_{\text{as}}\,\bar{A}_k + w_a\,\overline{\mathbf{u}_k^2}. \tag{12}$$

A barra indica média sobre os 15 músculos. $\bar{A}_k$ é o uso normalizado da assistência, nulo na execução final. A soma em j percorre as 8 juntas rastreadas, com pesos $\omega_j = 1$ para quadril, joelho e tornozelo dos dois lados e $\omega_j = 0{,}3$ para as aduções.

Por que multiplicativa. Com penalidades subtraídas, a recompensa por passo podia ficar negativa, e o agente aprendeu que cair cedo era melhor que continuar andando. Na forma tarefa $\cdot\, e^{-P}$, todo passo vivo vale $r_k \geq 0$; só o passo de queda é negativo. Perder os passos futuros é sempre pior. O termo $\overline{\mathbf{e}^3}$ aproxima a fadiga muscular, e o termo de variação penaliza excitações bruscas.

Tabela 5: Pesos e escalas da recompensa (`gaitConfig.m`, versão final).

Termo	Peso	Escala	Observação
pose	$w_p = 0{,}45$	$k_p = 3$	rad^{-2}
velocidade articular	$w_v = 0{,}10$	$k_v = 0{,}05$	$(\mathrm{rad/s})^{-2}$
velocidade de marcha	$w_s = 0{,}45$	$k_s = 2{,}5$	até o bloco 93: $w_s = 0{,}25$, $k_s = 1{,}5$
postura ereta	$w_u = 0{,}15$	$k_\theta = 20$, $k_h = 50$	inclinação e altura da pelve
viver	$w_{\mathrm{alive}} = 0{,}30$	—	bônus constante
esforço $\overline{e^3}$	$w_e = 0{,}2$		
variação de excitação	$w_\Delta = 2{,}0$		
assistência	$w_{\mathrm{as}} = 0{,}2$		zero sem assistência
magnitude da ação	$w_a = 0{,}02$		
queda	-2		no passo terminal

A recompensa máxima por passo é $0{,}45 + 0{,}10 + 0{,}45 + 0{,}15 + 0{,}30 = 1{,}45$, ou seja, no máximo cerca de 870 por episódio de 600 passos. No fim do treino, os episódios alcançaram cerca de 650.

7 Agente SAC e estrutura das redes neurais

7.1 Algoritmo

O agente é um `rlSACAgent` (Soft Actor-Critic) com política gaussiana e dois críticos Q, cada um com sua rede-alvo. Os críticos minimizam

$$J_Q(\theta_i) = \mathbb{E}\Big[\big(Q_{\theta_i}(\mathbf{o},\mathbf{a}) - y\big)^2\Big], \quad y = r + \gamma(1-d)\Big(\min_{i=1,2} Q_{\bar{\theta}_i}(\mathbf{o}',\mathbf{a}') - \alpha \log \pi_\phi(\mathbf{a}'|\mathbf{o}')\Big),\ \mathbf{a}' \sim \pi_\phi(\cdot|\mathbf{o}'). \tag{13}$$

O ator minimiza $J_\pi(\phi) = \mathbb{E}\big[\alpha \log \pi_\phi(\mathbf{a}|\mathbf{o}) - \min_i Q_{\theta_i}(\mathbf{o},\mathbf{a})\big]$. O peso de entropia α é ajustado para manter a entropia perto do alvo $\bar{\mathcal{H}} = -15$ (um por dimensão de ação). As redes-alvo seguem $\bar{\theta} \leftarrow \tau\theta + (1-\tau)\bar{\theta}$, com $\tau = 0{,}005$. A ação é amostrada como

$$\mathbf{a} = \tanh\big(\boldsymbol{\mu}_\phi(\mathbf{o}) + \boldsymbol{\sigma}_\phi(\mathbf{o}) \odot \boldsymbol{\varepsilon}\big), \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0},\mathbf{I}), \tag{14}$$

e escalada para os limites $[-1, 1]$. Nas avaliações a política é determinística: $\mathbf{a} = \tanh(\boldsymbol{\mu}_\phi(\mathbf{o}))$.

7.2 Arquitetura

As redes são as padrão do toolbox, criadas por `rlAgentInitializationOptions('NumHiddenUnit',256)`. A estrutura abaixo foi extraída do agente final (`results/best_agent.mat`). Todas as camadas são totalmente conectadas (FC) com ativação ReLU.

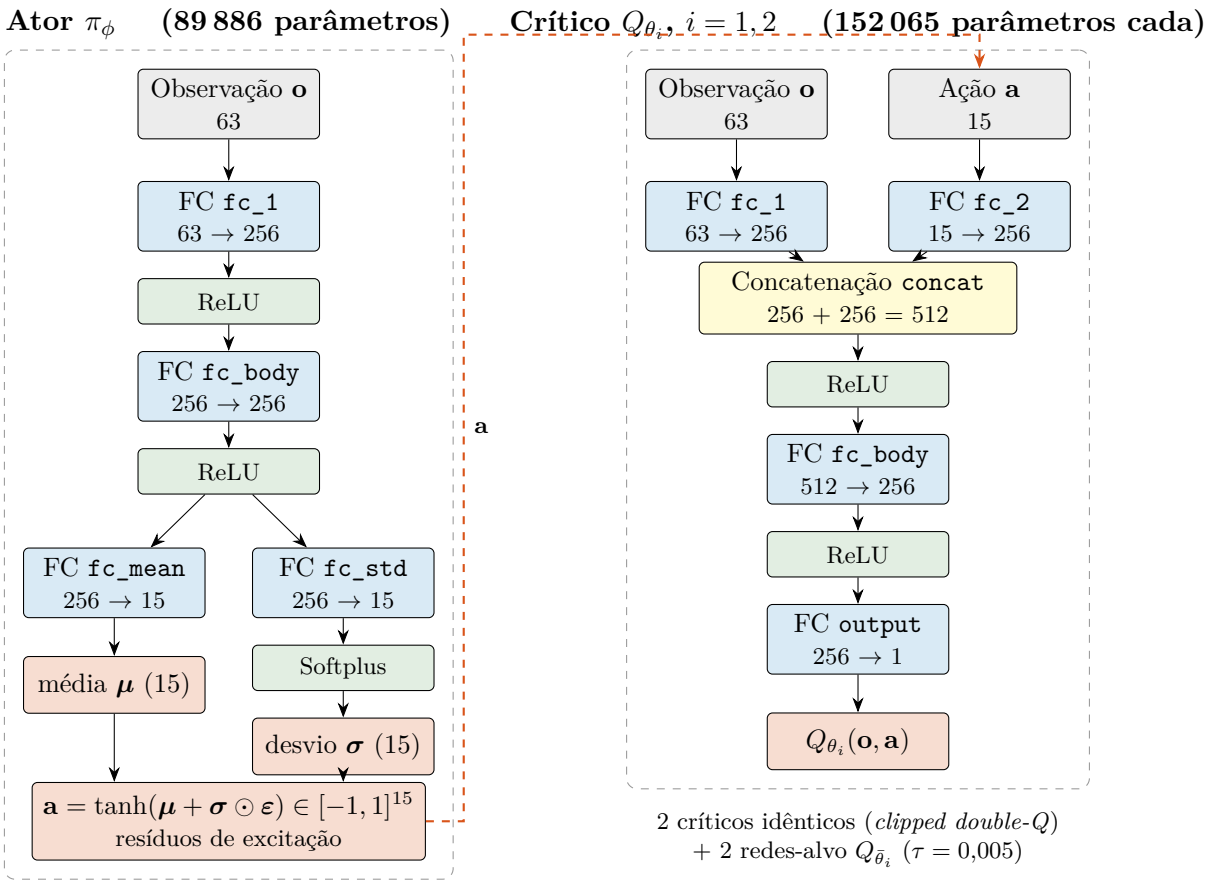


Figura 2: Estrutura das redes do agente SAC. O ator tem um tronco comum de duas camadas de 256 neurônios e duas cabeças: média e desvio-padrão (Softplus garante $\sigma > 0$). Cada crítico processa a observação e a ação em ramos separados de 256 neurônios, concatena e passa por uma camada de 256 até um único valor Q. A seta tracejada indica que a ação do ator entra nos críticos durante a atualização do ator.

Tabela 6: Camadas e parâmetros (extraídos com `getModel` do agente final).

Rede	Camada	Dimensão	Parâmetros
Ator	`fc_1` + ReLU	63 → 256	16 384
	`fc_body` + ReLU	256 → 256	65 792
	`fc_mean`	256 → 15	3 855
	`fc_std` + Softplus	256 → 15	3 855
	total		**89 886**
Crítico (×2)	`fc_1` (observação)	63 → 256	16 384
	`fc_2` (ação)	15 → 256	4 096
	`concat` + ReLU	512	—
	`fc_body` + ReLU	512 → 256	131 328
	`output`	256 → 1	257
	total		**152 065**

7.3 Hiperparâmetros

Tabela 7: Opções do agente (`train_sac.m`).

Parâmetro	Valor
Passo de amostragem	0,01 s
Fator de desconto γ	0,99
Mini-lote	256
Buffer de experiência	10^6 transições (não é salvo no checkpoint; zera a cada reinício)
Passos de aquecimento	2000
Frequência de aprendizado	1 atualização de gradiente a cada 4 passos
Otimizador	Adam; gradiente limitado a norma 1
Taxa de aprendizado (ator e críticos)	3×10^{-4} até o bloco 111; 1×10^{-4} depois
Suavização das redes-alvo τ	0,005
Entropia	alvo -15; peso inicial 0,05; taxa 3×10^{-4}

8 Procedimento de treino

`train_sac.m` treina em *blocos* de 40 episódios. Cada bloco chama `train` do toolbox com 4 workers em modo assíncrono. Ao fim de cada bloco:

- o histórico (recompensa média e máxima, duração dos episódios, tempo) vai para `results/train_log.csv`;
- o agente é salvo em `results/sac_checkpoint.mat`, e o treino pode ser retomado com `'resume',true`;
- a cada 3 blocos, a política determinística é avaliada sem assistência por 10 s (`evaluate_policy`); se ficar de pé o tempo todo, o agente é guardado em `results/agent_walk_chunkNNN.mat`.

8.1 Histórico das execuções

Tabela 8: Etapas do treino até o agente final.

Etapa	Blocos	Episódio	Velocidade (w_s, k_s)	Taxa	Resultado
Execução 1 (arquivada)	≈15	3 s	—	$3 \cdot 10^{-4}$	Currículo com assistência na pelve. A política aprendeu a desligar músculos e a exploração colapsou; descartada (`results/run1_assist_curriculum`).
Execução 2a	1–93	3 s	(0,25, 1,5)	$3 \cdot 10^{-4}$	Sem assistência. Avaliações de 5 s em pé a partir do bloco 27, mas marcha lenta (0,4–0,65 m/s) e muito assimétrica.
Execução 2b	94–114	6 s	(0,45, 2,5)	$3 \cdot 10^{-4}$	Primeiros 10 s em pé (blocos 102 e 111), mas a política oscilava entre andar e cair (2–10 s).
Execução 2c	112–150	6 s	(0,45, 2,5)	$1 \cdot 10^{-4}$	Retomada do agente do bloco 111. Quase todas as avaliações com 10 s em pé; melhor agente no bloco 129.

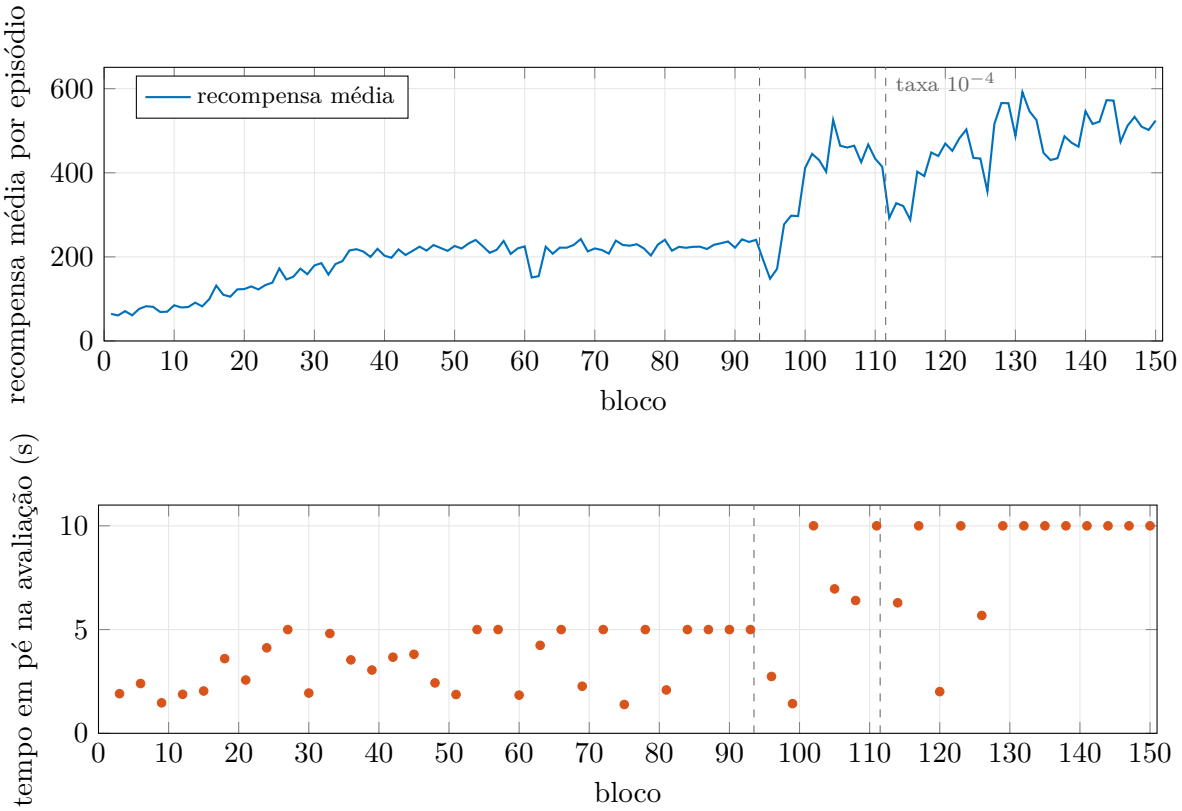


Figura 3: Curvas de treino da linhagem do agente final (`results/train_log.csv`). Até o bloco 93 a avaliação durava 5 s, e depois 10 s. A mudança de escala da recompensa no bloco 94 vem dos episódios duas vezes mais longos.

9 Avaliação e seleção do agente

9.1 Métricas

`evaluate_policy.m` simula a política determinística sem assistência e calcula:

- **tempo em pé** até a queda ou o fim da simulação;
- **velocidade média** $(x_{\text{pelve}}(T) - x_{\text{pelve}}(0))/T$;
- **RMSE e correlação** de cada junta (quadril, joelho e tornozelo, dos dois lados) contra a referência saudável na mesma fase;
- **assimetria de apoio** $100\,|\bar{s}_R - \bar{s}_L| / \frac{1}{2}(\bar{s}_R + \bar{s}_L)$, com $s = 1$ quando a força vertical do pé passa de 50 N;
- **índice de jerk**: RMS da terceira derivada dos ângulos, normalizado pela amplitude;
- **plausibilidade**: não cair, $0{,}9 < v < 1{,}5$ m/s, RMSE $< 10°$, correlação $> 0{,}8$ e assimetria $< 15\,\%$.

9.2 Comparação robusta

Uma única avaliação de 10 s a partir da fase 0 é ruidosa. Por isso `compare_agents.m` testa cada agente salvo por 20 s a partir de 4 fases iniciais (0; 0,25; 0,5; 0,75), em paralelo. Os agentes são ordenados pelo número de fases completadas, depois pelo tempo médio em pé e por fim pela correlação.

Tabela 9: Comparação dos 12 agentes salvos (`results/agent_comparison.csv`): médias das 4 fases, 20 s cada.

Agente (bloco)	Fases completas	Tempo (s)	v (m/s)	RMSE (°)	Corr.	Assim. (%)
129	**4/4**	**20,0**	**0,94**	**11,3**	**0,70**	**11,4**
102	4/4	20,0	0,63	10,8	0,68	29,2
150	4/4	20,0	0,73	11,6	0,63	26,3
135	4/4	20,0	0,72	11,5	0,61	29,7
147	4/4	20,0	0,67	11,0	0,55	31,6
117	3/4	15,5	0,86	13,2	0,59	27,5
144	3/4	15,2	0,86	13,1	0,52	22,5
132	3/4	15,2	0,91	12,9	0,57	33,4
141	3/4	15,2	1,00	12,3	0,60	30,9
123	3/4	15,2	0,86	12,9	0,56	37,0
111	2/4	11,7	0,92	13,1	0,63	25,4
138	0/4	11,2	0,66	14,4	0,50	39,0

10 Resultados do melhor agente

O agente do bloco 129 (`results/best_agent.mat`) foi o único que completou as 4 fases e ficou mais perto da marcha saudável em velocidade e simetria.

Tabela 10: Critérios de plausibilidade do melhor agente (médias das 4 fases).

Critério	Meta	Obtido	Atende?
Não cair	20 s em pé	20 s nas 4 fases	sim
Velocidade	0,9–1,5 m/s	0,94 m/s	sim
Assimetria de apoio	$< 15\,\%$	11,4 %	sim
RMSE articular	$< 10°$	11,3°	não
Correlação com a referência	$> 0{,}8$	0,70	não

A Figura 4 mostra uma avaliação de 20 s. O quadril do coto acompanha bem a referência (RMSE 8°). O joelho protético dobra no balanço, mas fica estendido demais no apoio, efeito da mola fraca ($k = 5$ N m/rad). O joelho intacto, com 20°, é a maior fonte de erro: dobra menos que o normal no balanço. A velocidade oscila dentro do passo entre ≈0,6 e 1,2 m/s, sem cair ao longo do tempo, e as excitações musculares têm padrão periódico e regular.

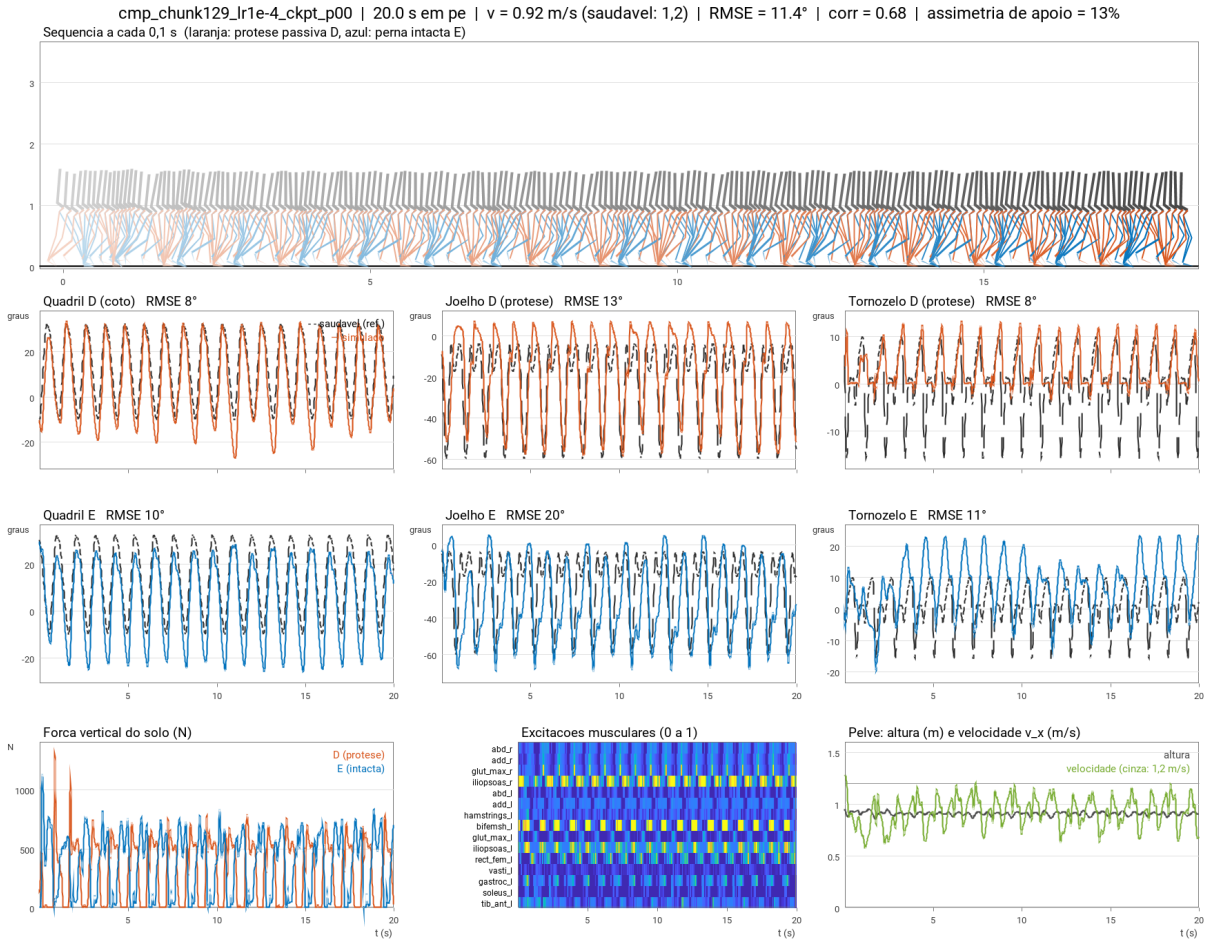


Figura 4: Melhor agente (bloco 129), 20 s a partir da fase 0. Topo: sequência a cada 0,1 s (laranja: prótese; azul: perna intacta). Meio: ângulos articulares contra a referência saudável (tracejado). Base: força vertical do solo, excitações musculares e altura e velocidade da pelve.

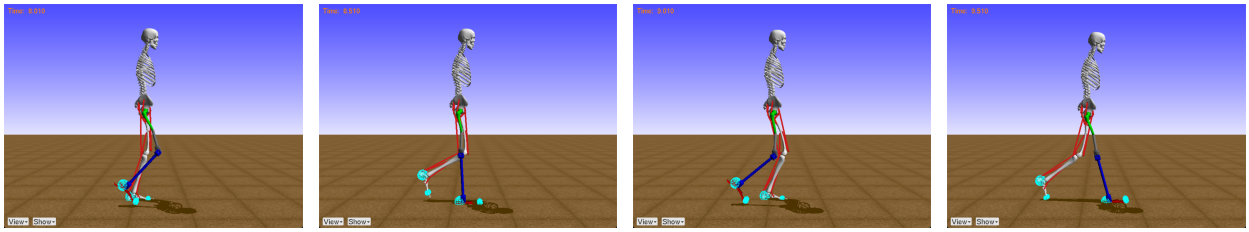


Figura 5: Quadros do visualizador 3D do OpenSim (Simbody) em $t = 8{,}0$; 8,5; 9,0 e 9,5 s: ossos, prótese (encaixe verde, pilão cinza e azul) e músculos (vermelho). Vídeo completo em `results/cmp_chunk129_lr1e-4_ckpt_p00_opensim.mp4`.

11 Comparação com a marcha saudável por ciclo

11.1 Dados saudáveis e procedimento

A referência de variabilidade vem do conjunto público de Fukuchi, Fukuchi e Duarte [1]: 42 adultos saudáveis (24 jovens e 18 idosos), marcha no solo em três velocidades, com ângulos e cinética já normalizados no ciclo (101 pontos, média por sujeito). Os dados estão no Figshare sob licença CC BY 4.0. Foram baixados os arquivos em texto (`WBDSascii.zip`) e a tabela de metadados (`WBDSinfo.csv`), guardados em `data/wbds/`.

- **Velocidade pareada.** O agente anda a 0,94 m/s. Por isso a faixa saudável principal é a da velocidade *lenta* do dataset, $0{,}86 \pm 0{,}13$ m/s. A média da velocidade *confortável*, $1{,}27 \pm 0{,}16$ m/s, aparece como linha tracejada.
- **Saudável.** Para cada sujeito, média das pernas direita e esquerda; depois média ±1 desvio-padrão entre os 42 sujeitos. Ângulos no plano sagital (flexão de quadril e joelho positivas, dorsiflexão positiva). Forças divididas pelo peso corporal (PC).
- **Simulação.** Melhor agente (bloco 129), 4 simulações de 20 s com fases iniciais 0; 0,25; 0,5 e 0,75 (`results/best_p*_traj.mat`). O primeiro segundo é descartado. Cada passada vai de um contato inicial (força vertical do pé acima de 20 N) ao seguinte do mesmo pé e é normalizada em 101 pontos. Isso dá 65 passadas de cada lado. O sinal do joelho do OpenSim (flexão negativa) é invertido, e as forças são divididas pelo peso do modelo (67,8 kg).
- **Código.** `src/gait_cycle_comparison.m` gera `docs/figs/ciclo_comparacao.csv`, que as Figuras 6 e 7 leem diretamente.

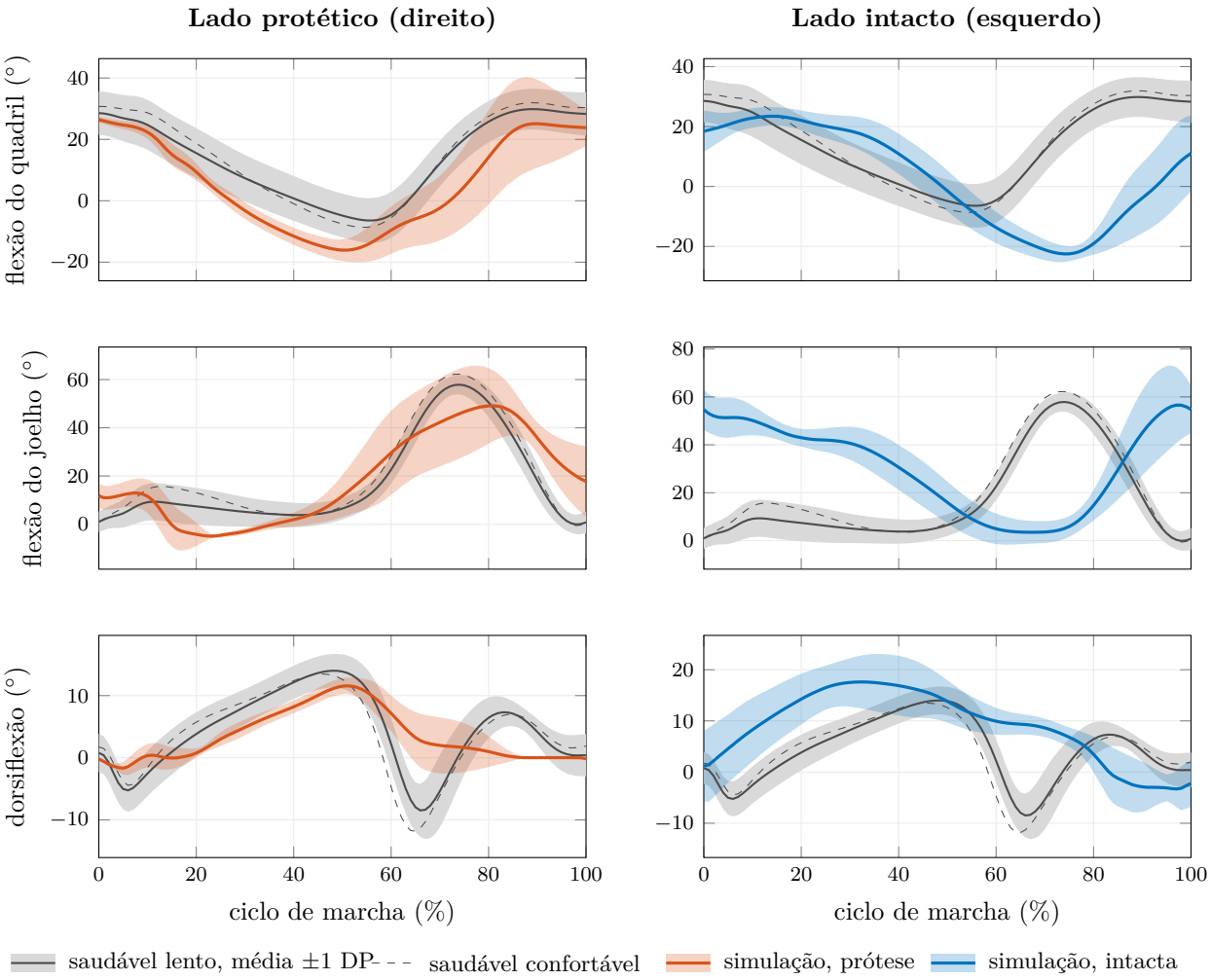


Figura 6: Ângulos articulares sagitais por ciclo de marcha (0 % = contato inicial do pé do mesmo lado). Faixa cinza: marcha saudável lenta, média ±1 DP, 42 sujeitos [1]. Tracejado: média saudável em velocidade confortável. Cores: média ±1 DP das 65 passadas simuladas de cada lado. No lado protético, o joelho e o tornozelo são as juntas passivas da prótese.

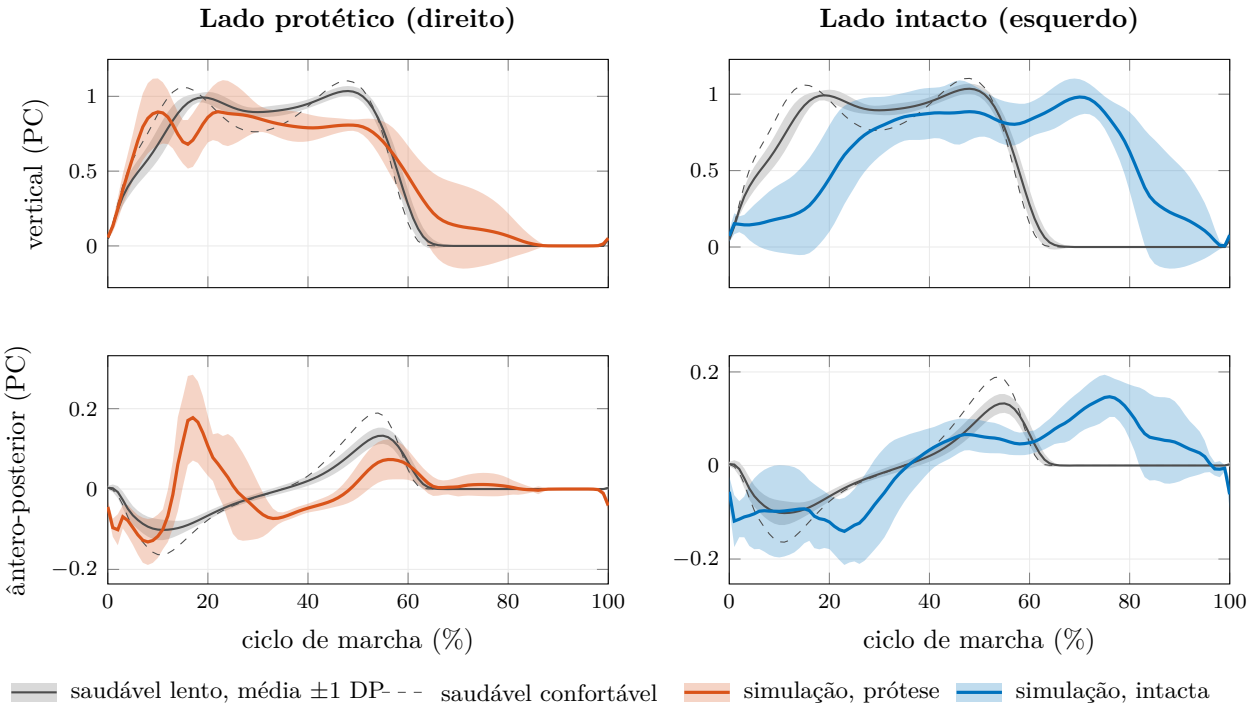


Figura 7: Forças de reação do solo por ciclo de marcha, em múltiplos do peso corporal (PC). Força ântero-posterior positiva = propulsão (para a frente). Mesma legenda da Figura 6.

11.2 Resultados

Tabela 11: Simulação contra a média saudável lenta (`docs/figs/ciclo_resumo.csv`). "Dentro da faixa" é a fração do ciclo em que a média simulada fica a até 1 DP da média saudável.

	Lado protético			Lado intacto		
Variável	RMSE	r	dentro da faixa	RMSE	r	dentro da faixa
Quadril (flexão)	9,1°	0,97	49 %	21,9°	0,13	34 %
Joelho (flexão)	10,1°	0,85	35 %	36,2°	−0,70	6 %
Tornozelo (dorsiflexão)	4,4°	0,71	48 %	8,4°	0,43	24 %
GRF vertical	0,14 PC	0,96	16 %	0,48 PC	0,26	3 %
GRF ântero-posterior	0,08 PC	0,19	11 %	0,07 PC	0,62	24 %
Duração do apoio	70,3 ± 7,3 % do ciclo			88,2 ± 6,4 % do ciclo		

Nos saudáveis, o apoio dura 64,3 ± 1,5 % do ciclo na velocidade lenta e 62,4 ± 1,2 % na confortável.

Lado protético. A forma geral segue a da marcha saudável.

- **Quadril do coto** ($r = 0{,}97$): mesmo formato, mas a extensão máxima é maior e chega antes, $\approx -16°$ perto de 50 % do ciclo, contra $\approx -6°$ perto de 56 % no saudável.
- **Joelho protético** ($r = 0{,}85$): fica quase estendido durante o apoio, encostando no batente perto de 20 %. No balanço dobra até $\approx 50°$, abaixo dos $\approx 60°$ saudáveis, e o pico vem mais tarde.
- **Tornozelo protético** ($r = 0{,}71$): acompanha a dorsiflexão do apoio, mas não faz a flexão plantar do pré-balanço e fica perto de 0° no balanço, como esperado de um pé passivo rígido ($k = 350$ N m/rad).

- **Força vertical** ($r = 0{,}96$): tem os dois picos. O descarregamento é lento e o apoio se estende até $\approx 70\,\%$ do ciclo.
- **Força ântero-posterior** ($r = 0{,}19$): tem um pico positivo atípico perto de 15 % e uma fase de propulsão menor perto de 50 %, com $\approx 0{,}08$ contra $\approx 0{,}15$ PC. A prótese passiva gera pouco impulso no fim do apoio.

Lado intacto. É onde a marcha mais se afasta da saudável. O pé esquerdo fica no ar só ≈0,16 s por passada e volta a tocar o solo cedo, com pouca carga (35–150 N), ainda com o joelho dobrado ≈55°. Só depois recebe o peso do corpo. Por isso o apoio da perna intacta dura 88 % do ciclo, e o ciclo, que começa nesse toque leve, fica desalinhado da referência: o joelho começa flexionado, o que dá correlação negativa. Na prática, a perna intacta compensa a prótese: fica mais tempo no chão e sustenta o corpo, um padrão comum em amputados transfemorais, mas aqui exagerado.

Diferença em relação à assimetria da Tabela 10. A assimetria de apoio de 11 % calculada por `evaluate_policy` usa limiar de 50 N e conta tempo total de contato; os toques leves de 20–50 N do pé intacto ficam de fora. Com o limiar de 20 N e a definição por passada desta seção, a diferença de apoio entre os lados é bem maior (70 contra 88 % do ciclo).

12 Visualização

Nesta máquina, os gráficos do MATLAB travam tanto no modo `-batch` quanto ao abrir o desktop por linha de comando. Por isso a visualização foi feita de dois modos que não dependem deles:

- `make_media.m`: usa o OpenSim para a cinemática direta (posição de quadris, joelhos, tornozelos e esferas de contato a cada amostra) e desenha a figura-resumo e o vídeo de boneco diretamente em pixels com `insertShape` e `insertText`. Grava `<tag>_resultado.png`, `<tag>_marcha.mp4` e `<tag>_pontos.csv`.
- `render_opensim.m` + `capture_window.ps1`: reproduz a trajetória no visualizador 3D do OpenSim (Simbody), com a câmera seguindo a pelve. Cada quadro é capturado da janela por um script PowerShell, que mantém a janela sempre por cima. Quadros cobertos por outra janela (céu não azul) são capturados de novo. O resultado é `<tag>_opensim.mp4`.
- Os arquivos `<tag>_motion.mot` podem ser abertos no OpenSim GUI (*File* → *Load Motion*).

13 Desempenho computacional

- Bloco de 40 episódios: ≈430 s com episódios de 3 s e 600–900 s com episódios de 6 s (4 workers, CPU de 12 núcleos lógicos).
- O aprendiz roda no processo principal e é o gargalo. Com uma atualização por passo, um bloco levava ≈920 s; com uma a cada 4 passos, caiu para 150–250 s (episódios de 3 s).
- **GPU:** `bench_gpu.m` mediu o custo do aprendiz sem a simulação, usando um ambiente falso com as mesmas dimensões. O resultado foi 177 ms por atualização na CPU e 188 ms numa RTX 2060. Com redes de 256 neurônios e lote de 256, o tempo vai no custo fixo de cada chamada, não no cálculo. A opção `'device','gpu'` existe em `train_sac`, mas não traz ganho. A simulação (OpenSim + `ode23`) roda só na CPU.

14 Problemas encontrados e soluções

Problema	Solução
JVM `EXCEPTION_ACCESS_VIOLATION` em `Coordinate.getValue`	O `State` devolvido pelo `Manager` era liberado junto com ele. Copiar o estado e, depois, eliminar o `Manager`.
"State Cache entry out of date"	Chamar `realizeDynamics` após `setY`.
`getStateVariableDerivativeValues` indisponível	Usar `getY`/`getYDot` do Simbody.
Thelen2003 sem `ignore_tendon_compliance`	Opção removida.
Recompensa negativa incentivava cair	Forma multiplicativa tarefa $\cdot e^{-P}$ (sempre ≥ 0 enquanto vivo).
Termo de velocidade sem gradiente parado	Núcleo mais largo (k_s de 5 para 1,5) e mais peso; depois, com a marcha estável, $k_s = 2{,}5$ e $w_s = 0{,}45$.
Aprendiz como gargalo (922 s por bloco)	`LearningFrequency=4`.
`equilibrateMuscles` falhava em algumas fases	Sortear outra fase (até 6 vezes) e tratar estado inválido como queda.
Execução com currículo de assistência degenerou	Reinício sem assistência.
Horizonte de 3 s saturado (avaliações paradas em 3–5 s)	Episódios de 6 s e mais peso na velocidade.
Política oscilando entre blocos	Taxa de aprendizado de $3 \cdot 10^{-4}$ para 10^{-4}, retomando do melhor agente.
Gráficos do MATLAB travam em `-batch`	Desenho direto em pixels (`make_media`) e captura do visualizador Simbody (`render_opensim`).
`setWindowTitle` inexistente na API MATLAB do visualizador	Chamada removida.
Janela sobreposta durante a gravação	Janela `HWND_TOPMOST` e verificação de cada quadro.

15 Limitações e próximos passos

- **RMSE e correlação** ainda fora da meta, principalmente pelos joelhos. O joelho protético é uma mola e um amortecedor lineares. Um modelo com freio de apoio ou atrito dependente da carga representaria melhor joelhos passivos comerciais e deixaria o apoio mais estável.
- **Velocidade** de 0,94 m/s, ainda abaixo dos 1,2 m/s da referência. Amputados transfemorais costumam andar mais devagar, então talvez a referência deva ser ajustada para essa população.
- **Marcha planar**: o próximo estágio é liberar as coordenadas 3D (`gaitConfig('planar',false)`).
- **Referência aproximada**: curvas reconstruídas de pontos-chave, não de dados medidos de um sujeito.
- **Buffer não persistente**: cada reinício do treino perde o replay buffer, e os primeiros blocos pioram.

16 Guia de uso

```
cd src
% treino (retoma de results/sac_checkpoint.mat se existir)
train_sac('nChunks', 150, 'episodesPerChunk', 40, 'workers', 4, 'resume', true, 'lr', 1e-4)
% avaliacao de um agente
```

```
S = load('../results/best_agent.mat');
M = evaluate_policy(S.agent, 'assist', 0, 'duration', 20, 'initPhase', 0, 'tag', 'final')
% comparacao robusta de todos os agentes salvos
T = compare_agents('duration', 20, 'phases', [0 .25 .5 .75], 'workers', 6)
% figura-resumo e video de boneco; video no visualizador 3D do OpenSim
make_media('final')
render_opensim('final')
```

Referências

[1] C. A. Fukuchi, R. K. Fukuchi e M. Duarte. A public dataset of overground and treadmill walking kinematics and kinetics in healthy individuals. *PeerJ*, 6:e4640, 2018. `https://doi.org/10.7717/peerj.4640`. Dados: `https://doi.org/10.6084/m9.figshare.5722711` (CC BY 4.0).

A Referência dos arquivos

Arquivo	Função
`src/gaitConfig.m`	Todos os parâmetros: modelo, integrador, músculos, coordenadas, ganhos, prótese, assistência, término e recompensa. Também calcula $F_{\max}$ e a máscara músculo × junta.
`src/healthyGaitReference.m`	Curvas periódicas de marcha saudável (quadril, joelho, tornozelo, adução).
`src/ProstheticGaitEnv.m`	Ambiente de RL: monta o modelo (prótese passiva, assistência, controlador), `reset` com RSI, `step` com integração `ode23`, controlador-base, observação, recompensa e gravação da trajetória.
`src/train_sac.m`	Cria ou retoma o agente SAC, treina em blocos em paralelo, avalia, salva checkpoints e histórico.
`src/evaluate_policy.m`	Simulação determinística, métricas, `.mot`, `_traj.mat` e `_traj.csv`.
`src/compare_agents.m`	Avaliação robusta de vários agentes em várias fases iniciais.
`src/gait_cycle_comparison.m`	Segmenta as passadas simuladas, normaliza no ciclo e compara com a marcha saudável do dataset público (média ±1 DP).
`data/wbds/`	Dataset público de marcha saudável [1] (arquivos em texto processados em `proc/`).
`src/make_media.m`	Figura-resumo e vídeo de boneco desenhados em pixels.
`src/render_opensim.m`	Vídeo no visualizador 3D do OpenSim.
`src/capture_window.ps1`	Captura da janela do visualizador (PowerShell, user32/GDI).
`src/bench_gpu.m`	Custo do aprendiz SAC em CPU e GPU.
`src/test_deriv.m`	Tempo de avaliação de $\dot{\mathbf{y}}$ e comparação de integradores.
`src/test_baseline.m`, `src/sweep.m`	Testes do controlador-base com ação zero e varredura de parâmetros.
`results/`	Checkpoints, agentes salvos, `train_log.csv`, `train_console.log`, avaliações, figuras e vídeos.

B Código-fonte

B.1 `gaitConfig.m`

```
function c = gaitConfig(varargin)
%GAITCONFIG Parametros do ambiente, controlador-base e recompensa.
%   c = gaitConfig('planar', false, 'assistDecayEpisodes', 3000, ...)

here = fileparts(mfilename('fullpath'));
c.modelFile = fullfile(here, '..', 'OpenSIM', 'OS4_gait14dof15musc_2Act_RTFP_DynAct.osim');

c.planar = true;      % estagio 1: marcha restrita ao plano sagital (+ add/abd do quadril)
c.dt = 0.01;          % passo de controle (s)
c.maxSteps = 600;     % 6 s por episodio (~5.5 ciclos; era 3 s ate o bloco 93)
c.odeSolver = @ode23;   % integrador do MATLAB (ode23 foi o mais rapido; ode15s/ode23t p/ casos rigidos)
c.odeRelTol = 1e-3; c.odeAbsTol = 1e-4;
c.momentArmEvery = 3;

c.muscles = {'abd_r','add_r','glut_max_r','iliopsoas_r', ...
             'abd_l','add_l','hamstrings_l','bifemsh_l','glut_max_l','iliopsoas_l', ...
             'rect_fem_l','vasti_l','gastroc_l','soleus_l','tib_ant_l'};

c.allCoords = {'pelvis_tilt','pelvis_list','pelvis_rotation','pelvis_tx','pelvis_ty','pelvis_tz', ...
               'hip_flexion_r','hip_adduction_r','hip_rotation_r','knee_angle_r','ankle_angle_r', ...
               'hip_flexion_l','hip_adduction_l','knee_angle_l','ankle_angle_l'};

% juntas controladas pelos musculos (controlador-base)
c.ctrlJoints = {'hip_flexion_r','hip_adduction_r','hip_flexion_l','hip_adduction_l','knee_angle_l','
    ankle_angle_l'};
c.trackJoints = {'hip_flexion_r','knee_angle_r','ankle_angle_r','hip_flexion_l','knee_angle_l','ankle_angle_l
    ','hip_adduction_r','hip_adduction_l'};
c.trackW      = [1 1 1 1 1 1 0.3 0.3];

c.targetSpeed = 1.2;      % m/s
c.pelvisTiltRef = 0;      % rad
c.initPhase = 0;
c.randomInitPhase = true;
c.initPenetration = 0.004;

% ganhos do controlador-base (Nm/rad, Nm s/rad)
c.Kp = [250 150 250 150 200 150];
c.Kd = [ 15  10  15  10  12   8];
c.KpTorso = 1500; c.KdTorso = 100;
c.lambda = 1e-4; c.baseExcitation = 0.02;

% PROTESE PASSIVA (sem controle): mola torcional + amortecedor viscoso
% joelho: tipo mecanico com fricc. hidraulica simples + leve mola de extensao
% tornozelo: pe de armazenamento de energia (ESAR), ~6 Nm/grau p/ ~75 kg
c.prosthesis.knee  = struct("coord","knee_angle_r",  "stiffness", 5,   "rest", 0, "damping", 3);
c.prosthesis.ankle = struct("coord","ankle_angle_r", "stiffness", 350, "rest", 0, "damping", 2);

% acao do agente
c.residualScale = 0.25;

c.actionFilter = 0.5;     % u = f*u_prev + (1-f)*a

% assistencia na pelve (curriculo) -- removida gradualmente
c.assistCoords = {'pelvis_tilt','pelvis_tx','pelvis_ty','pelvis_list'};
c.assistKp = [400 0 3000 300];
c.assistKd = [40 300 300 30];
c.assistNorm = [100 100 300 100];
c.assistStart = 0;
c.assistDecayEpisodes = 0;   % 0 = assistencia fixa em c.assistStart

% terminacao
c.minPelvisHeight = 0.70;
c.maxTilt = 0.6;
c.fallPenalty = 2;

% recompensa (imitacao + tarefa + regularizacao fisiologica)
c.wPose = 0.45; c.kPose = 3;
c.wVel = 0.10;  c.kVel = 0.05;
c.wSpeed = 0.45; c.kSpeed = 2.5;   % mais peso e kernel mais estreito: 0.6 m/s ja e claramente pior que 1.2 (
    era 0.25/1.5 ate o bloco 93)
c.wUpright = 0.15; c.kUpright = 20; c.kHeight = 50;
c.wAlive = 0.30;
c.wEffort = 0.2; c.wSmooth = 2.0; c.wAssist = 0.2; c.wAction = 0.02;

```

```
for i = 1:2:numel(varargin), c.(varargin{i}) = varargin{i+1}; end

if c.planar
    c.lockedCoords = {'pelvis_list','pelvis_rotation','pelvis_tz','hip_rotation_r'};
    keep = ~ismember(c.assistCoords, c.lockedCoords);
    c.assistCoords = c.assistCoords(keep); c.assistKp = c.assistKp(keep);
    c.assistKd = c.assistKd(keep); c.assistNorm = c.assistNorm(keep);
else
    c.lockedCoords = {'hip_rotation_r'};
end
c.obsCoords = setdiff(c.allCoords, [c.lockedCoords, {'hip_rotation_l','lumbar_extension'}], 'stable');

% Fmax e mascara musculo x junta
c.Fmax = muscleFmax(c);
c.muscleJointMask = muscleJointMask(c);
end

function F = muscleFmax(c)
import org.opensim.modeling.*
persistent cache
if isempty(cache)
    m = Model(c.modelFile); ms = m.getMuscles();
    cache = containers.Map();
    for i = 0:ms.getSize()-1, cache(char(ms.get(i).getName())) = ms.get(i).getMaxIsometricForce(); end
end
F = cellfun(@(n) cache(n), c.muscles);
end

function M = muscleJointMask(c)
% quais musculos cruzam quais juntas (anatomia do modelo)
map = struct( ...
  'hip_flexion_r',   {{'abd_r','add_r','glut_max_r','iliopsoas_r'}}, ...
  'hip_adduction_r', {{'abd_r','add_r','glut_max_r','iliopsoas_r'}}, ...
  'hip_flexion_l',   {{'abd_l','add_l','hamstrings_l','glut_max_l','iliopsoas_l','rect_fem_l'}}, ...
  'hip_adduction_l', {{'abd_l','add_l','hamstrings_l','glut_max_l','iliopsoas_l','rect_fem_l'}}, ...
  'knee_angle_l',    {{'hamstrings_l','bifemsh_l','rect_fem_l','vasti_l','gastroc_l'}}, ...
  'ankle_angle_l',   {{'gastroc_l','soleus_l','tib_ant_l'}});
M = false(numel(c.ctrlJoints), numel(c.muscles));
for j = 1:numel(c.ctrlJoints)
    M(j,:) = ismember(c.muscles, map.(c.ctrlJoints{j}));
end
end
```

B.2 healthyGaitReference.m

```
function ref = healthyGaitReference()
%HEALTHYGAITREFERENCE Cinematica sagital normativa de marcha saudavel.
%   Curvas aproximadas (Winter 2009 / Perry 1992), velocidade ~1.2 m/s,
%   ciclo ~1.1 s. Fase 0 = contato inicial do membro em questao.
%   Convencoes OpenSim (gait2354): hip_flexion + = flexao,
%   knee_angle - = flexao, ankle_angle + = dorsiflexao. Saida em rad.
%
%   ref.eval(phase) -> struct com q e qd (por unidade de fase) de
%   hip, knee, ankle e hip_add.

% [ % do ciclo , angulo (graus) ]
hip = [0 30; 10 25; 20 15; 30 5; 40 -4; 50 -10; 55 -9; 60 -3; 70 12; 80 26; 87 32; 94 31; 100 30];
kneeFlex = [0 4; 5 11; 15 18; 25 13; 35 6; 42 4; 50 8; 56 20; 62 38; 68 55; 73 60; 80 53; 87 32; 94 10; 100
     4];
ankle = [0 0; 5 -5; 10 -3; 20 3; 30 7; 45 10; 50 7; 55 -3; 61 -16; 65 -14; 72 -5; 80 1; 90 1; 100 0];
hipAdd = [0 0; 10 4; 20 5; 40 2; 55 -2; 65 -5; 80 -3; 95 -1; 100 0];

N = 1000;
ph = (0:N-1)'/N;
ref.phase = ph;
ref.hip    = deg2rad(periodicFit(hip, ph));
ref.knee   = -deg2rad(periodicFit(kneeFlex, ph));   % convencao OpenSim
ref.ankle  = deg2rad(periodicFit(ankle, ph));
ref.hipAdd = deg2rad(periodicFit(hipAdd, ph));
ref.N = N;
ref.cycleTime = 1.1;     % s
ref.speed = 1.2;         % m/s
```

```
end

function y = periodicFit(kp, ph)
% Interpolacao pchip periodica seguida de suavizacao por serie de Fourier
x = kp(:,1)/100; v = kp(:,2);
xx = [x(1:end-1)-1; x(1:end-1); x+1];
vv = [v(1:end-1); v(1:end-1); v];
yd = pchip(xx, vv, ph);
% manter 8 harmonicos -> curva suave e periodica
Y = fft(yd); K = 8;
Y(K+2:end-K) = 0;
y = real(ifft(Y));
end
```

B.3 ProstheticGaitEnv.m

```
classdef ProstheticGaitEnv < rl.env.MATLABEnvironment
    %PROSTHETICGAITENV Ambiente de RL: dinamica direta OpenSim de um
    %   amputado transfemoral (OS4_gait14dof15musc_2Act_RTFP_DynAct.osim).
    %
    %   Acao (15): residuos sobre um controlador-base de rastreamento
    %       1..15 -> excitacoes dos musculos (ordem de cfg.muscles)
    %   Protese PASSIVA: joelho e tornozelo sem controle, apenas mola +
    %   amortecedor (SpringGeneralizedForce) e os batentes do modelo.
    %   O agente aprende correcoes para que a marcha imite a referencia
    %   saudavel (estilo DeepMimic) com excitacoes suaves e baixo esforco.

    properties
        cfg
        ref
        assist = 1          % nivel atual da assistencia na pelve (0..1)
        episodeCount = 0
        record = false      % gravar trajetoria (avaliacao)
        traj = struct()
    end

    properties (Transient)
        model; state; ctrlFns; coords; muscles; contactForces
        actIdx; assistIdx
        built = false
    end

    properties (Access = private)
        t = 0; k = 0; phase0 = 0
        u_prev; e_prev
        R           % matriz de bracos de momento (nJoint x nMus)
        ty0 = 0.94
        bodyWeight = 1
    end

    methods
        function this = ProstheticGaitEnv(cfg)
            if nargin < 1, cfg = gaitConfig(); end
            nObs = ProstheticGaitEnv.obsSize(cfg);
            oi = rlNumericSpec([nObs 1]);
            oi.Name = 'gait_obs';
            ai = rlNumericSpec([numel(cfg.muscles) 1], 'LowerLimit', -1, 'UpperLimit', 1);
            ai.Name = 'residual_excitations';
            this = this@rl.env.MATLABEnvironment(oi, ai);
            this.cfg = cfg;
            this.ref = healthyGaitReference();
        end

        %% ---------------- construcao do modelo ----------------
        function build(this)
            import org.opensim.modeling.*
            c = this.cfg;
            m = Model(c.modelFile);
            m.setUseVisualizer(false);
            cs = m.getCoordinateSet();
            for n = c.lockedCoords
                cs.get(n{1}).setDefaultLocked(true);
```

```
            cs.get(n{1}).setDefaultValue(0);
        end
        % PROTESE PASSIVA: desliga os atuadores ativos do joelho/tornozelo
        % e adiciona mola + amortecedor em cada junta protetica
        % (os batentes KneeLimit_r / AnkleLimit_r do modelo continuam).
        for n = {'knee_actuator','ankle_actuator'}
            m.updForceSet().get(n{1}).set_appliesForce(false);
        end
        P = c.prosthesis;
        for n = fieldnames(P)'
            pp = P.(n{1});
            sp = SpringGeneralizedForce(pp.coord);
            sp.setName(['passive_' n{1}]);
            sp.setStiffness(pp.stiffness); sp.setRestLength(pp.rest); sp.setViscosity(pp.damping);
            m.addForce(sp);
        end
        % atuadores de assistencia na pelve ("mao de Deus", curriculo)
        for n = c.assistCoords
            ca = CoordinateActuator(n{1});
            ca.setName(['assist_' n{1}]);
            ca.setOptimalForce(1);
            ca.setMinControl(-1e5); ca.setMaxControl(1e5);
            m.addForce(ca);
        end
        brain = PrescribedController();
        acts = m.getActuators();
        nA = acts.getSize();
        names = cell(nA,1);
        for i = 0:nA-1
            a = acts.get(i);
            names{i+1} = char(a.getName());
            brain.addActuator(a);
            brain.prescribeControlForActuator(names{i+1}, Constant(0));
        end
        m.addController(brain);
        s = m.initSystem();

        fs = brain.get_ControlFunctions();
        this.ctrlFns = cell(nA,1);
        for i = 0:nA-1, this.ctrlFns{i+1} = Constant.safeDownCast(fs.get(i)); end
        % mapeamento acao -> indice de atuador
        order = c.muscles;
        this.actIdx = cellfun(@(x) find(strcmp(names, x)), order);
        this.assistIdx = cellfun(@(x) find(strcmp(names, ['assist_' x])), c.assistCoords);

        this.coords = struct();
        for n = c.allCoords
            this.coords.(n{1}) = cs.get(n{1});
        end
        ms = m.getMuscles();
        this.muscles = cell(numel(c.muscles),1);
        for i = 1:numel(c.muscles), this.muscles{i} = ms.get(c.muscles{i}); end
        this.contactForces = {HuntCrossleyForce.safeDownCast(m.getForceSet().get('foot_r')), ...
                              HuntCrossleyForce.safeDownCast(m.getForceSet().get('foot_l'))};
        this.bodyWeight = m.getTotalMass(s) * 9.80665;
        this.model = m; this.state = s;
        this.built = true;
    end

    %% ---------------- reset ----------------
    function obs = reset(this)
        if ~this.built, this.build(); end
        c = this.cfg; m = this.model;
        s = m.initializeState();
        this.episodeCount = this.episodeCount + 1;
        if c.assistDecayEpisodes > 0
            this.assist = c.assistStart * max(0, 1 - this.episodeCount / c.assistDecayEpisodes);
        else
            this.assist = c.assistStart;
        end
        % Inicializacao por estado de referencia (RSI). Se o equilibrio
        % musculo-tendao nao convergir nessa fase, sorteia outra.
        for attempt = 1:6
            if c.randomInitPhase || attempt > 1, this.phase0 = rand; else, this.phase0 = c.initPhase; end
            s = m.initializeState();
```

```
            if this.initPose(s, attempt < 6), break; end
        end
        this.state = s; this.t = 0; this.k = 0;
        this.u_prev = zeros(numel(c.muscles),1);
        this.e_prev = 0.05*ones(numel(c.muscles),1);
        this.R = this.momentArms(s);
        if this.record, this.traj = struct('t',[],'q',[],'e',[],'grf',[],'grf3',[],'r',[],'assist',[]);
 end
        obs = this.observe(s);
    end
end

methods (Access = private)
    function ok = initPose(this, s, strict)
        % coloca o modelo na pose de referencia da fase this.phase0;
        % ok = false se o equilibrio dos musculos falhar (strict)
        c = this.cfg; m = this.model; ok = true;
        [q, qd] = this.refAt(this.phase0);
        jn = fieldnames(q);
        for i = 1:numel(jn)
            this.coords.(jn{i}).setValue(s, q.(jn{i}), false);
            this.coords.(jn{i}).setSpeedValue(s, qd.(jn{i}));
        end
        this.coords.pelvis_tilt.setValue(s, c.pelvisTiltRef, false);
        this.coords.pelvis_tx.setValue(s, 0, false);
        this.coords.pelvis_tx.setSpeedValue(s, c.targetSpeed);
        % altura da pelve: pe mais baixo apenas tocando o solo
        this.coords.pelvis_ty.setValue(s, 1.0, false);
        m.realizePosition(s);
        this.coords.pelvis_ty.setValue(s, 1.0 - this.lowestContact(s) - c.initPenetration, true);
        this.ty0 = this.coords.pelvis_ty.getValue(s);
        % ativacoes iniciais e equilibrio musculo-tendao
        for i = 1:numel(this.muscles)
            this.muscles{i}.setActivation(s, 0.05);
        end
        try
            m.equilibrateMuscles(s);
        catch
            % ultimo recurso: segue com comprimentos de fibra padrao
            ok = ~strict;
        end
    end
end

methods

    %% ---------------- step ----------------
    function [obs, r, done, info] = step(this, action)
        c = this.cfg; s = this.state; m = this.model;
        a = max(-1, min(1, double(action(:))));
        % filtro passa-baixa nas acoes -> excitacoes mais suaves
        u = c.actionFilter * this.u_prev + (1 - c.actionFilter) * a;
        phase = this.currentPhase();
        eBase = this.baseline(s, phase);
        ex = min(1, max(0.01, eBase + c.residualScale * u));
        for i = 1:numel(ex), this.ctrlFns{this.actIdx(i)}.setValue(ex(i)); end
        aF = this.assistForces(s, phase);
        for i = 1:numel(aF), this.ctrlFns{this.assistIdx(i)}.setValue(aF(i)); end

        failed = false;
        try
            % Integracao da ODE inteiramente no MATLAB. O OpenSim so
            % avalia ydot = f(t, y, u) (u constante no intervalo: ZOH).
            y0 = s.getY().getAsMat();
            opts = odeset('RelTol', c.odeRelTol, 'AbsTol', c.odeAbsTol, 'Refine', 1);
            [~, Y] = c.odeSolver(@(tt, yy) this.dynamics(s, tt, yy), [this.t, this.t + c.dt], y0, opts);
            s.setTime(this.t + c.dt);
            s.setY(org.opensim.modeling.Vector.createFromMat(Y(end,:)'));
            m.realizeDynamics(s);
            if any(~isfinite(Y(end,:))), failed = true; end
        catch
            failed = true;
        end
        this.t = this.t + c.dt; this.k = this.k + 1;
        this.state = s;
```

```
            if mod(this.k, c.momentArmEvery) == 0 && ~failed
                this.R = this.momentArms(s);
            end

            try
                obs = this.observe(s);
                [r, terms] = this.reward(s, ex, u, aF);
                ty = this.coords.pelvis_ty.getValue(s);
                tilt = this.coords.pelvis_tilt.getValue(s);
                fell = ty < c.minPelvisHeight || abs(tilt - c.pelvisTiltRef) > c.maxTilt;
            catch
                % estado invalido (ex.: integracao divergiu): encerra o episodio
                obs = zeros(this.getObservationInfo().Dimension); r = 0; fell = true; failed = true;
                terms = struct('pose',0,'vel',0,'speed',0,'upright',0,'effort',0,'smooth',0,'assist',0);
            end
            if failed || any(~isfinite(obs)), fell = true; obs(~isfinite(obs)) = 0; end
            if fell, r = r - c.fallPenalty; end
            done = fell || this.k >= c.maxSteps;
            this.u_prev = u; this.e_prev = ex;
            info = terms;
            if this.record
                this.traj.t(end+1,1) = this.t;
                this.traj.q(end+1,:) = this.coordVector(s);
                this.traj.e(end+1,:) = ex';
                this.traj.grf(end+1,:) = this.grf(s)';
                this.traj.grf3(end+1,:) = this.grf3(s);   % [Fx Fy Fz]_dir [Fx Fy Fz]_esq (N)
                this.traj.r(end+1,1) = r;
                this.traj.assist(end+1,:) = aF';
            end
        end
    end

    methods
        function yd = dynamics(this, s, t, y)
            %DYNAMICS Lado direito da ODE: y = [q; u; z] (Simbody), z =
            %ativacoes e comprimentos de fibra dos musculos + ativacao dos
            %atuadores da protese. Avaliado pelo OpenSim, integrado no MATLAB.
            s.setTime(t);
            s.setY(org.opensim.modeling.Vector.createFromMat(y));
            this.model.realizeAcceleration(s);
            yd = s.getYDot().getAsMat();
        end
    end

    methods (Access = private)
        function ph = currentPhase(this)
            ph = mod(this.phase0 + this.t / this.ref.cycleTime, 1);
        end

        function [q, qd] = refAt(this, phase)
            % referencia para a perna esquerda (intacta) na fase phase e
            % para a direita (protese) defasada de meio ciclo
            R_ = this.ref; T = R_.cycleTime;
            L = phase; Rr = mod(phase + 0.5, 1);
            f = @(sig, p) interpPeriodic(sig, p);
            df = @(sig, p) (interpPeriodic(sig, mod(p+1e-3,1)) - interpPeriodic(sig, mod(p-1e-3,1))) / (2e-3*
 T);
            q.hip_flexion_l = f(R_.hip, L);    qd.hip_flexion_l = df(R_.hip, L);
            q.knee_angle_l  = f(R_.knee, L);   qd.knee_angle_l  = df(R_.knee, L);
            q.ankle_angle_l = f(R_.ankle, L);  qd.ankle_angle_l = df(R_.ankle, L);
            q.hip_flexion_r = f(R_.hip, Rr);   qd.hip_flexion_r = df(R_.hip, Rr);
            q.knee_angle_r  = f(R_.knee, Rr);  qd.knee_angle_r  = df(R_.knee, Rr);
            q.ankle_angle_r = f(R_.ankle, Rr); qd.ankle_angle_r = df(R_.ankle, Rr);
            q.hip_adduction_l = f(R_.hipAdd, L);  qd.hip_adduction_l = df(R_.hipAdd, L);
            q.hip_adduction_r = f(R_.hipAdd, Rr); qd.hip_adduction_r = df(R_.hipAdd, Rr);
        end

        function h = lowestContact(this, s)
            import org.opensim.modeling.*
            bs = this.model.getBodySet();
            spheres = {'foot_r',[-0.03 -0.005 0],0.05; 'foot_r',[0.15 -0.03 -0.026],0.025; ...
                       'foot_r',[0.15 -0.03 0.026],0.025; 'calcn_l',[0.03 0.02 0],0.05; ...
                       'toes_l',[0.02 -0.005 -0.026],0.025; 'toes_l',[0.02 -0.005 0.026],0.025};
            h = inf;
            for i = 1:size(spheres,1)
```

```
            p = bs.get(spheres{i,1}).findStationLocationInGround(s, Vec3(spheres{i,2}(1), spheres{i,2}(2)
, spheres{i,2}(3)));
            h = min(h, p.get(1) - spheres{i,3});
        end
    end

    function R = momentArms(this, s)
        jn = this.cfg.ctrlJoints;
        R = zeros(numel(jn), numel(this.muscles));
        this.model.realizePosition(s);
        for j = 1:numel(jn)
            cj = this.coords.(jn{j});
            for i = 1:numel(this.muscles)
                if this.cfg.muscleJointMask(j,i)
                    R(j,i) = this.muscles{i}.computeMomentArm(s, cj);
                end
            end
        end
    end

    function e = baseline(this, s, phase)
        %Controlador-base: PD no espaco articular -> torques desejados ->
        %excitacoes por minimos quadrados nao-negativos (tipo otimizacao
        %estatica). So musculos; a protese e passiva.
        c = this.cfg;
        [qr, qdr] = this.refAt(mod(phase + c.dt/this.ref.cycleTime, 1));
        jn = c.ctrlJoints; nJ = numel(jn);
        tau = zeros(nJ,1);
        tilt = this.coords.pelvis_tilt.getValue(s) - c.pelvisTiltRef;
        tiltd = this.coords.pelvis_tilt.getSpeedValue(s);
        g = this.grf(s) / this.bodyWeight;     % [vert_r; vert_l]
        for j = 1:nJ
            cj = this.coords.(jn{j});
            tau(j) = c.Kp(j) * (qr.(jn{j}) - cj.getValue(s)) + c.Kd(j) * (qdr.(jn{j}) - cj.getSpeedValue(
s));
        end
        % equilibrio do tronco pelo quadril de apoio (estilo SIMBICON)
        jr = strcmp(jn, 'hip_flexion_r'); jl = strcmp(jn, 'hip_flexion_l');
        wR = g(1) / max(g(1) + g(2), 1e-3); wL = 1 - wR;
        if g(1) + g(2) < 0.05, wR = 0; wL = 0; end
        torso = c.KpTorso * tilt + c.KdTorso * tiltd;
        tau(jr) = tau(jr) + wR * torso;
        tau(jl) = tau(jl) + wL * torso;
        % musculos: min ||F e - tau||^2 + lambda ||e||^2, e >= 0
        F = this.R .* (c.Fmax(:)');
        A = [F; sqrt(c.lambda) * diag(c.Fmax)];
        b = [tau; zeros(numel(c.Fmax),1)];
        e = lsqnonneg(A, b);
        e = min(1, e + c.baseExcitation);
        % protese passiva: nenhum controle sobre joelho/tornozelo direitos
    end

    function f = assistForces(this, s, phase)
        c = this.cfg; f = zeros(numel(c.assistCoords),1);
        if this.assist <= 0, return; end
        for i = 1:numel(c.assistCoords)
            n = c.assistCoords{i}; cj = this.coords.(n);
            switch n
                case 'pelvis_tx'
                    f(i) = c.assistKd(i) * (c.targetSpeed - cj.getSpeedValue(s));
                case 'pelvis_ty'
                    f(i) = max(0, c.assistKp(i) * (this.ty0 - cj.getValue(s)) - c.assistKd(i) * cj.
getSpeedValue(s));
                case 'pelvis_tilt'
                    f(i) = c.assistKp(i) * (c.pelvisTiltRef - cj.getValue(s)) - c.assistKd(i) * cj.
getSpeedValue(s);
                otherwise
                    f(i) = -c.assistKp(i) * cj.getValue(s) - c.assistKd(i) * cj.getSpeedValue(s);
            end
        end
        f = this.assist * f;
    end

    function g = grf(this, s)
        this.model.realizeDynamics(s);
```

```
            g = zeros(2,1);
            for i = 1:2
                v = this.contactForces{i}.getRecordValues(s);
                g(i) = -v.get(1);    % componente vertical sobre o pe
            end
            g = max(0, g);
        end

        function g = grf3(this, s)
            % forca de reacao do solo SOBRE cada pe (N), eixos do ground:
            % X antero-posterior (+ para frente), Y vertical, Z medio-lateral
            g = zeros(1, 6);
            for i = 1:2
                v = this.contactForces{i}.getRecordValues(s);   % 0..2 = forca sobre o solo
                g(3*i-2:3*i) = -[v.get(0) v.get(1) v.get(2)];
            end
        end

        function obs = observe(this, s)
            c = this.cfg;
            q = []; qd = [];
            for n = c.obsCoords
                cj = this.coords.(n{1});
                if ~strcmp(n{1}, 'pelvis_tx'), q(end+1,1) = cj.getValue(s); end %#ok<AGROW>
                qd(end+1,1) = cj.getSpeedValue(s); %#ok<AGROW>
            end
            act = zeros(numel(this.muscles),1);
            for i = 1:numel(this.muscles), act(i) = this.muscles{i}.getActivation(s); end
            ph = this.currentPhase();
            [qr, ~] = this.refAt(mod(ph + c.dt/this.ref.cycleTime, 1));
            qref = cellfun(@(n) qr.(n), c.trackJoints)';
            g = this.grf(s) / this.bodyWeight;
            obs = [q; 0.1*qd; act; g; sin(2*pi*ph); cos(2*pi*ph); qref; this.u_prev];
        end

        function [r, terms] = reward(this, s, ex, u, aF)
            c = this.cfg; ph = this.currentPhase();
            [qr, qdr] = this.refAt(ph);
            ep = 0; ev = 0;
            for j = 1:numel(c.trackJoints)
                n = c.trackJoints{j}; cj = this.coords.(n);
                ep = ep + c.trackW(j) * (cj.getValue(s) - qr.(n))^2;
                ev = ev + c.trackW(j) * (cj.getSpeedValue(s) - qdr.(n))^2;
            end
            vx = this.coords.pelvis_tx.getSpeedValue(s);
            tilt = this.coords.pelvis_tilt.getValue(s) - c.pelvisTiltRef;
            ty = this.coords.pelvis_ty.getValue(s);
            terms.pose  = exp(-c.kPose * ep);
            terms.vel   = exp(-c.kVel * ev);
            terms.speed = exp(-c.kSpeed * (vx - c.targetSpeed)^2);
            terms.upright = exp(-c.kUpright * tilt^2 - c.kHeight * (ty - this.ty0)^2);
            terms.effort = mean(ex.^3);                          % ~ fadiga muscular
            terms.smooth = mean((ex - this.e_prev).^2);          % variacao de excitacao
            terms.assist = min(1, mean(abs(aF) ./ c.assistNorm(:)));
            % forma multiplicativa: recompensa sempre >= 0, logo cair (perder
            % os passos futuros) e sempre pior do que continuar andando
            task = c.wPose*terms.pose + c.wVel*terms.vel + c.wSpeed*terms.speed + ...
                   c.wUpright*terms.upright + c.wAlive;
            pen = c.wEffort*terms.effort + c.wSmooth*terms.smooth + ...
                  c.wAssist*terms.assist + c.wAction*mean(u.^2);
            r = task * exp(-pen);
        end

        function v = coordVector(this, s)
            n = this.cfg.allCoords; v = zeros(1, numel(n));
            for i = 1:numel(n), v(i) = this.coords.(n{i}).getValue(s); end
        end
    end

    methods (Static)
        function n = obsSize(cfg)
            n = (numel(cfg.obsCoords) - 1) + numel(cfg.obsCoords) + numel(cfg.muscles) ...
                + 2 + 2 + numel(cfg.trackJoints) + numel(cfg.muscles);
        end
    end
```

```
end

function y = interpPeriodic(sig, p)
N = numel(sig); x = p * N; i0 = floor(x); w = x - i0;
i0 = mod(i0, N) + 1; i1 = mod(i0, N) + 1;
y = (1 - w) * sig(i0) + w * sig(i1);
end
```

B.4 train_sac.m

```
function train_sac(varargin)
%TRAIN_SAC Treina um agente SAC que modula as excitacoes musculares e os
%   atuadores da protese. Treino em blocos: entre blocos a assistencia na
%   pelve e reduzida (curriculo), o agente e salvo e avaliado sem assistencia.
%
%   train_sac('nChunks',60,'episodesPerChunk',40,'workers',4,'resume',true)

p = inputParser;
p.addParameter('nChunks', 150);
p.addParameter('episodesPerChunk', 40);
p.addParameter('workers', 4);
p.addParameter('resume', true);
p.addParameter('assistInit', 0);         % 0 = sem assistencia (recomendado: o sinal de queda guia o
    aprendizado)
p.addParameter('assistFullChunks', 0);
p.addParameter('assistStepDown', 0.05);  % reducao da assistencia por bloco bem-sucedido
p.addParameter('assistStepUp', 0);       % >0 reativa assistencia se cair cedo
p.addParameter('evalEvery', 3);
p.addParameter('learnEvery', 4);
p.addParameter('lr', []);                % [] = manter a do agente (3e-4 no inicio)
p.addParameter('device', 'cpu');         % 'gpu' = redes do aprendiz na GPU. Medido (bench_gpu.m, RTX 2060):
                                         % 177 ms/gradiente na CPU vs 188 na GPU -> sem ganho com redes de 256
p.parse(varargin{:}); o = p.Results;

here = fileparts(mfilename('fullpath'));
outDir = fullfile(here, '..', 'results'); if ~exist(outDir, 'dir'), mkdir(outDir); end
ckpt = fullfile(outDir, 'sac_checkpoint.mat');
logFile = fullfile(outDir, 'train_log.csv');

cfg = gaitConfig();
env = ProstheticGaitEnv(cfg);
obsInfo = getObservationInfo(env); actInfo = getActionInfo(env);

startChunk = 1; history = [];
if o.resume && isfile(ckpt)
    S = load(ckpt); agent = S.agent; startChunk = S.chunk + 1; history = S.history;
    fprintf('Retomando do bloco %d\n', startChunk);
else
    init = rlAgentInitializationOptions('NumHiddenUnit', 256);
    agent = rlSACAgent(obsInfo, actInfo, init);
    ao = agent.AgentOptions;
    ao.SampleTime = cfg.dt;
    ao.DiscountFactor = 0.99;
    ao.MiniBatchSize = 256;
    ao.ExperienceBufferLength = 1e6;
    ao.TargetSmoothFactor = 5e-3;
    ao.NumWarmStartSteps = 2000;
    ao.LearningFrequency = 1;        % gradiente a cada passo
    ao.NumGradientStepsPerUpdate = 1;
    ao.ActorOptimizerOptions.LearnRate = 3e-4;
    for i = 1:numel(ao.CriticOptimizerOptions)
        ao.CriticOptimizerOptions(i).LearnRate = 3e-4;
        ao.CriticOptimizerOptions(i).GradientThreshold = 1;
    end
    ao.ActorOptimizerOptions.GradientThreshold = 1;
    ao.EntropyWeightOptions.TargetEntropy = -15;
    ao.EntropyWeightOptions.EntropyWeight = 0.05;
    agent.AgentOptions = ao;
end
% aprendiz no processo principal e o gargalo: 1 atualizacao a cada
% o.learnEvery passos (aplicado tambem ao retomar de checkpoint)
agent.AgentOptions.LearningFrequency = o.learnEvery;
```

```
agent.AgentOptions.ResetExperienceBufferBeforeTraining = false;   % manter buffer entre blocos
if ~isempty(o.lr)   % taxa de aprendizado (ator e criticos); menor = politica mais estavel
    ao = agent.AgentOptions;
    ao.ActorOptimizerOptions.LearnRate = o.lr;
    for i = 1:numel(ao.CriticOptimizerOptions), ao.CriticOptimizerOptions(i).LearnRate = o.lr; end
    agent.AgentOptions = ao;
    fprintf('Taxa de aprendizado = %g\n', o.lr);
end
actor = getActor(agent); actor.UseDevice = o.device; agent = setActor(agent, actor);
critics = getCritic(agent); for i = 1:numel(critics), critics(i).UseDevice = o.device; end
agent = setCritic(agent, critics);
fprintf('Aprendiz em: %s\n', o.device);

if o.workers > 0
    pool = gcp('nocreate');
    if isempty(pool) || pool.NumWorkers ~= o.workers
        delete(pool); pool = parpool('Processes', o.workers);
    end
    addAttachedFiles(pool, {fullfile(here, 'ProstheticGaitEnv.m'), fullfile(here, 'gaitConfig.m'), fullfile(here, 'healthyGaitReference.m')});
    parfevalOnAll(pool, @addpath, 0, here);
end

% curriculo ADAPTATIVO de assistencia: so reduz quando o agente aguenta
% >= 80% do episodio; aumenta um pouco se cair antes de 50%.
if isempty(history), a = o.assistInit; else, a = history(end, 2); end
for chunk = startChunk:o.nChunks
    if ~isempty(history) && chunk > o.assistFullChunks
        fracSurv = history(end, 5) / cfg.maxSteps;
        if fracSurv >= 0.8,     a = max(0, a - o.assistStepDown);
        elseif fracSurv < 0.5,  a = min(1, a + o.assistStepUp);
        end
        if a < 0.03, a = 0; end
    end
    env.cfg.assistStart = a; env.cfg.assistDecayEpisodes = 0;

    topts = rlTrainingOptions( ...
        'MaxEpisodes', o.episodesPerChunk, ...
        'MaxStepsPerEpisode', cfg.maxSteps, ...
        'ScoreAveragingWindowLength', 20, ...
        'StopTrainingCriteria', 'none', ...
        'Plots', 'none', 'Verbose', false, ...
        'UseParallel', o.workers > 0);
    if o.workers > 0, topts.ParallelizationOptions.Mode = 'async'; end

    tic;
    stats = train(agent, env, topts);
    el = toc;
    R = stats.EpisodeReward; L = stats.EpisodeSteps;
    row = [chunk, a, mean(R), max(R), mean(L), max(L), el, NaN, NaN, NaN];
    history = [history; row]; %#ok<AGROW>
    fprintf('[%s] bloco %d/%d | assist=%.2f | R med=%.1f max=%.1f | passos med=%.0f max=%d | %.0f s\n', ...
        datestr(now, 'HH:MM:SS'), chunk, o.nChunks, a, mean(R), max(R), mean(L), max(L), el);

    if mod(chunk, o.evalEvery) == 0 || chunk == o.nChunks
        M = evaluate_policy(agent, 'assist', 0, 'duration', 10, 'tag', sprintf('chunk%03d', chunk), 'quiet', true);
        fprintf('   AVAL (sem assistencia): %.2f s em pe | v=%.2f m/s | RMSE=%.1f graus | corr=%.2f | simetria=%.0f%% | jerk=%.2f\n', ...
            M.survivalTime, M.meanSpeed, M.rmseMean, M.corrMean, M.stanceSymmetry, M.jerkIndex);
        history(end, 8:10) = [M.survivalTime, M.rmseMean, M.corrMean];
        if M.survivalTime >= 0.99*10, save(fullfile(outDir, sprintf('agent_walk_chunk%03d.mat', chunk)), 'agent', 'M'); end
    end
    save(ckpt, 'agent', 'chunk', 'history', 'cfg');
    writematrix(history, logFile);
end
end
```

B.5 evaluate_policy.m

```
function M = evaluate_policy(agent, varargin)
```

```
%EVALUATE_POLICY Simula a politica (deterministica) e mede a qualidade da marcha.
%   M = evaluate_policy(agent, 'assist', 0, 'duration', 10, 'tag', 'final')
%   agent = [] -> apenas o controlador-base (acao zero).
%   Gera: results/<tag>_motion.mot (abrir no OpenSim GUI sobre o modelo),
%         results/<tag>_gait.png e retorna metricas.
p = inputParser;
p.addParameter('assist', 0); p.addParameter('duration', 10);
p.addParameter('tag', 'eval'); p.addParameter('quiet', false);
p.addParameter('initPhase', 0);
p.parse(varargin{:}); o = p.Results;

here = fileparts(mfilename('fullpath'));
outDir = fullfile(here, '..', 'results'); if ~exist(outDir, 'dir'), mkdir(outDir); end
cfg = gaitConfig('assistStart', o.assist, 'randomInitPhase', false, 'initPhase', o.initPhase, ...
                 'maxSteps', round(o.duration / 0.01));
env = ProstheticGaitEnv(cfg); env.record = true;
obs = reset(env);
if ~isempty(agent)
    try, agent.UseExplorationPolicy = false; catch, end
end
for k = 1:cfg.maxSteps
    if isempty(agent), a = zeros(15,1); else, a = getAction(agent, {obs}); a = a{1}; end
    [obs, ~, done] = step(env, a);
    if done, break; end
end
tr = env.traj; M = gaitMetrics(tr, cfg, env.ref);
M.tag = o.tag;
writeMot(fullfile(outDir, [o.tag '_motion.mot']), tr, cfg);
save(fullfile(outDir, [o.tag '_traj.mat']), 'tr', 'M', 'cfg');
% graficos so no MATLAB desktop (em -batch os graficos derrubam o MATLAB nesta maquina)
if usejava('desktop'), plotGait(fullfile(outDir, [o.tag '_gait.png']), tr, cfg, env.ref, M); end
writematrix([tr.t tr.q tr.grf tr.e], fullfile(outDir, [o.tag '_traj.csv']));
if ~o.quiet, disp(M); end
end

function M = gaitMetrics(tr, cfg, ref)
t = tr.t; q = tr.q; n = numel(t);
M.survivalTime = t(end);
M.meanSpeed = (q(end,4) - q(1,4)) / max(t(end) - t(1), eps);
% fase da referencia em cada amostra (perna esquerda)
ph = mod(cfg.initPhase + t / ref.cycleTime, 1);
idx = [7 10 11 12 14 15];   % hipR kneeR ankR hipL kneeL ankL
sig = {ref.hip, ref.knee, ref.ankle, ref.hip, ref.knee, ref.ankle};
off = [0.5 0.5 0.5 0 0 0];
names = {'hipR','kneeR','ankR','hipL','kneeL','ankL'};
for j = 1:6
    r = interp1((0:ref.N)'/ref.N, [sig{j}; sig{j}(1)], mod(ph + off(j), 1));
    e = rad2deg(q(:,idx(j)) - r);
    M.rmse.(names{j}) = sqrt(mean(e.^2));
    if n > 5, cc = corrcoef(q(:,idx(j)), r); M.corr.(names{j}) = cc(1,2); else, M.corr.(names{j}) = NaN; end
end
M.rmseMean = mean(cell2mat(struct2cell(M.rmse)));
M.corrMean = mean(cell2mat(struct2cell(M.corr)));
% suavidade: jerk medio normalizado das juntas (rad/s^3 normalizado pela amplitude)
dt = cfg.dt;
if n > 10
    J = diff(q(:,idx), 3) / dt^3;
    amp = max(max(q(:,idx)) - min(q(:,idx)), 1e-3);
    M.jerkIndex = mean(sqrt(mean(J.^2)) ./ amp) * (ref.cycleTime^3) / 1e3;
else
    M.jerkIndex = NaN;
end
% simetria do tempo de apoio (pela GRF vertical)
st = tr.grf > 50;
M.stanceR = mean(st(:,1)); M.stanceL = mean(st(:,2));
M.stanceSymmetry = 100 * abs(M.stanceR - M.stanceL) / max(0.5*(M.stanceR + M.stanceL), eps);
M.peakGRF_BW = max(tr.grf) / (9.80665 * 75);
M.excitationChange = mean(abs(diff(tr.e(:,1:15))), 'all');
M.meanExcitation = mean(tr.e(:,1:15), 'all');
M.plausible = M.survivalTime >= 0.98*cfg.maxSteps*cfg.dt && M.meanSpeed > 0.9 && M.meanSpeed < 1.5 ...
    && M.rmseMean < 10 && M.corrMean > 0.8 && M.stanceSymmetry < 15;
end

function writeMot(file, tr, cfg)
names = cfg.allCoords; rot = ~ismember(names, {'pelvis_tx','pelvis_ty','pelvis_tz'});
```

```
D = tr.q; D(:,rot) = rad2deg(D(:,rot));
fid = fopen(file, 'w');
fprintf(fid, 'gait_rl\nversion=1\nnRows=%d\nnColumns=%d\ninDegrees=yes\nendheader\n', size(D,1), size(D,2)+1)
    ;
fprintf(fid, '%s', 'time'); fprintf(fid, '\t%s', names{:}); fprintf(fid, '\n');
fprintf(fid, [repmat('%.6f\t', 1, size(D,2)) '%.6f\n'], [tr.t D]');
fclose(fid);
end

function plotGait(file, tr, cfg, ref, M)
f = figure('Visible', 'off', 'Position', [100 100 1200 800]);
ph = mod(cfg.initPhase + tr.t / ref.cycleTime, 1);
idx = [7 10 11 12 14 15]; sig = {ref.hip, ref.knee, ref.ankle, ref.hip, ref.knee, ref.ankle};
off = [0.5 0.5 0.5 0 0 0]; ttl = {'Quadril D (residual)','Joelho D (protese)','Tornozelo D (protese)','
    Quadril E','Joelho E','Tornozelo E'};
for j = 1:6
    subplot(3,3,j);
    r = interp1((0:ref.N)'/ref.N, [sig{j}; sig{j}(1)], mod(ph + off(j), 1));
    plot(tr.t, rad2deg(r), 'k--', tr.t, rad2deg(tr.q(:,idx(j))), 'b', 'LineWidth', 1.2);
    title(ttl{j}); ylabel('graus'); grid on;
end
subplot(3,3,7); plot(tr.t, tr.grf); title('GRF vertical (N)'); legend('D','E'); grid on;
subplot(3,3,8); plot(tr.t, tr.e(:,1:15)); title('Excitacoes musculares'); ylim([0 1]); grid on;
subplot(3,3,9); plot(tr.t, tr.q(:,5), tr.t, tr.q(:,1)); title('Pelve: altura (m) e inclinacao (rad)'); grid
    on;
sgtitle(sprintf('%s | %.1f s | v=%.2f m/s | RMSE=%.1f graus | corr=%.2f | simetria=%.0f%%', ...
    strrep(M.tag,'_',' '), M.survivalTime, M.meanSpeed, M.rmseMean, M.corrMean, M.stanceSymmetry));
exportgraphics(f, file); close(f);
end
```

B.6 compare_agents.m

```
function T = compare_agents(varargin)
%COMPARE_AGENTS Avaliacao robusta dos agentes salvos: cada agente anda
%   'duration' segundos a partir de varias fases iniciais do ciclo. Escolhe o
%   mais robusto (nao o que teve sorte numa unica avaliacao).
%
%   T = compare_agents()                          % todos results/agent_walk_*.mat
%   T = compare_agents('duration', 20, 'phases', [0 .25 .5 .75], 'workers', 4)

p = inputParser;
p.addParameter('duration', 20);
p.addParameter('phases', [0 0.25 0.5 0.75]);
p.addParameter('workers', 4);
p.addParameter('files', {});
p.parse(varargin{:}); o = p.Results;

here = fileparts(mfilename('fullpath'));
outDir = fullfile(here, '..', 'results');
files = o.files;
if isempty(files)
    d = dir(fullfile(outDir, 'agent_walk_*.mat')); files = {d.name};
end
nA = numel(files); nP = numel(o.phases);
fprintf('%d agentes x %d fases x %g s\n', nA, nP, o.duration);

if o.workers > 0
    pool = gcp('nocreate');
    if isempty(pool), pool = parpool('Processes', o.workers); end
    addAttachedFiles(pool, {fullfile(here, 'ProstheticGaitEnv.m'), fullfile(here, 'gaitConfig.m'), ...
        fullfile(here, 'healthyGaitReference.m'), fullfile(here, 'evaluate_policy.m')});
    parfevalOnAll(pool, @addpath, 0, here);
end

jobs = [repelem((1:nA)', nP), repmat((1:nP)', nA, 1)];
R = nan(size(jobs, 1), 7);   % surv, v, rmse, corr, sym, jerk, plausible
parfor (j = 1:size(jobs, 1), o.workers)   % workers = 0 -> serial, sem abrir pool
    a = jobs(j, 1); ph = o.phases(jobs(j, 2));
    S = load(fullfile(outDir, files{a}), 'agent');
    [~, nm] = fileparts(files{a});
    tag = sprintf('cmp_%s_p%02d', erase(nm, 'agent_walk_'), round(100 * ph));
```

```
    M = evaluate_policy(S.agent, 'assist', 0, 'duration', o.duration, 'initPhase', ph, 'tag', tag, 'quiet', true);
    R(j, :) = [M.survivalTime, M.meanSpeed, M.rmseMean, M.corrMean, M.stanceSymmetry, M.jerkIndex, M.plausible];
end

T = table();
for a = 1:nA
    r = R(jobs(:, 1) == a, :);
    full = r(:, 1) >= 0.99 * o.duration;
    T(a, :) = table(string(files{a}), mean(r(:,1)), sum(full), mean(r(:,2)), mean(r(:,3)), mean(r(:,4)), ...
        mean(r(:,5)), mean(r(:,6)), sum(r(:,7)), ...
        'VariableNames', {'agente','tempoMedio_s','fasesCompletas','v_ms','RMSE_graus','corr','simetria_pct','jerk','plausivel'});
end
% robustez primeiro, depois qualidade da marcha
T = sortrows(T, {'fasesCompletas','tempoMedio_s','corr'}, {'descend','descend','descend'});
disp(T);
writetable(T, fullfile(outDir, 'agent_comparison.csv'));
writematrix([jobs R], fullfile(outDir, 'agent_comparison_raw.csv'));
end
```

B.7 gait_cycle_comparison.m

```
function S = gait_cycle_comparison(varargin)
%GAIT_CYCLE_COMPARISON Angulos articulares e forcas de reacao do solo da
%   simulacao, por ciclo de marcha (0-100 %), contra a marcha saudavel do
%   dataset publico de Fukuchi, Fukuchi & Duarte (2018, PeerJ 6:e4640;
%   figshare 10.6084/m9.figshare.5722711, CC BY 4.0): 42 adultos, marcha
%   no solo em velocidade lenta (S, 0,86 +/- 0,13 m/s) e confortavel
%   (C, 1,27 +/- 0,16 m/s).
%
%   Simulacao: cada passada vai de um contato inicial (forca vertical do pe
%   passa de 'thr' N) ao seguinte do mesmo pe; e normalizada em 101 pontos.
%   Saudavel: media de cada sujeito (perna D e E juntas) -> media +/- 1 DP
%   entre sujeitos.
%
%   S = gait_cycle_comparison('tags', {'best_p00','best_p25','best_p50','best_p75'})
%   Gera docs/figs/ciclo_comparacao.csv (para o pgfplots do relatorio) e
%   docs/figs/ciclo_resumo.csv.

p = inputParser;
p.addParameter('tags', {'best_p00','best_p25','best_p50','best_p75'});
p.addParameter('thr', 20);          % N, limiar de contato
p.addParameter('skip', 1.0);        % s iniciais descartados (transitorio)
p.parse(varargin{:}); o = p.Results;
import org.opensim.modeling.*

here = fileparts(mfilename('fullpath'));
root = fullfile(here, '..');
resDir = fullfile(root, 'results'); figDir = fullfile(root, 'docs', 'figs');
dsDir = fullfile(root, 'data', 'wbds', 'proc');
pct = (0:100)';
vars = {'hip','knee','ankle','grfv','grfap'};

%% ---------- marcha saudavel (dataset) ----------
H = struct();
for cls = 'SC'
    fa = dir(fullfile(dsDir, sprintf('WBDS*walk0%cang.txt', cls)));
    X = struct(); for v = vars, X.(v{1}) = []; end
    for i = 1:numel(fa)
        a = readtable(fullfile(fa(i).folder, fa(i).name), 'FileType', 'text', 'Delimiter', '\t');
        k = readtable(fullfile(fa(i).folder, strrep(fa(i).name, 'ang.txt', 'knt.txt')), 'FileType', 'text', 'Delimiter', '\t');
        g = 9.81;    % GRF do dataset em N/kg -> peso corporal (BW)
        X.hip(:,end+1)   = mean([a.RHipAngleZ   a.LHipAngleZ], 2, 'omitnan');
        X.knee(:,end+1)  = mean([a.RKneeAngleZ  a.LKneeAngleZ], 2, 'omitnan');
        X.ankle(:,end+1) = mean([a.RAnkleAngleZ a.LAnkleAngleZ], 2, 'omitnan');
        X.grfv(:,end+1)  = mean([k.RGRFY k.LGRFY], 2, 'omitnan') / g;
        X.grfap(:,end+1) = mean([k.RGRFX k.LGRFX], 2, 'omitnan') / g;
    end
    for v = vars
```

```
        Y = X.(v{1}); Y = Y(:, all(isfinite(Y), 1));
        H.(cls).(v{1}).m = mean(Y, 2); H.(cls).(v{1}).s = std(Y, 0, 2); H.(cls).(v{1}).n = size(Y, 2);
    end
end

%% ---------- simulacao: passadas normalizadas ----------
m = Model(gaitConfig().modelFile); st = m.initSystem();
BW = m.getTotalMass(st) * 9.81;
C = struct('R', struct(), 'L', struct());
for side = 'RL', for v = vars, C.(side).(v{1}) = []; end, end
stance = struct('R', [], 'L', []); nStr = struct('R', 0, 'L', 0);
for it = 1:numel(o.tags)
    T = load(fullfile(resDir, [o.tags{it} '_traj.mat'])); tr = T.tr;
    if ~isfield(tr, 'grf3') || isempty(tr.grf3), error('%s sem grf3: reavalie com o ambiente atual', o.tags{it}); end
    t = tr.t; q = rad2deg(tr.q);
    % colunas de allCoords: 7 hip_flexion_r, 10 knee_angle_r, 11 ankle_angle_r, 12 hip_flexion_l, 14 knee_angle_l, 15 ankle_angle_l
    sig.R = struct('hip', q(:,7),  'knee', -q(:,10), 'ankle', q(:,11), 'grfv', tr.grf3(:,2)/BW, 'grfap', tr.grf3(:,1)/BW);
    sig.L = struct('hip', q(:,12), 'knee', -q(:,14), 'ankle', q(:,15), 'grfv', tr.grf3(:,5)/BW, 'grfap', tr.grf3(:,4)/BW);
    for side = 'RL'
        fz = sig.(side).grfv * BW;
        on = find(fz(2:end) >= o.thr & fz(1:end-1) < o.thr) + 1;     % contatos iniciais
        on = on(t(on) >= o.skip);
        for j = 1:numel(on)-1
            i0 = on(j); i1 = on(j+1);
            if t(i1) - t(i0) < 0.5 || t(i1) - t(i0) > 2.0, continue; end   % descarta passadas espurias
            tt = (t(i0:i1) - t(i0)) / (t(i1) - t(i0)) * 100;
            for v = vars
                C.(side).(v{1})(:,end+1) = interp1(tt, sig.(side).(v{1})(i0:i1), pct);
            end
            off = find(fz(i0:i1) < o.thr, 1);                        % fim do apoio
            stance.(side)(end+1) = 100 * (off - 1) / (i1 - i0);
            nStr.(side) = nStr.(side) + 1;
        end
    end
end

%% ---------- tabela para o pgfplots e resumo ----------
% colunas: <grupo>_<variavel>_{m,lo,hi} com lo/hi = media -/+ 1 DP
% grupos: hS/hC = saudavel lento/confortavel; sR/sL = simulacao protese/intacta
cols = {'pct'}; D = pct;
for v = vars
    G = {};   % {nome, media, dp}
    for cls = 'SC', G(end+1,:) = {sprintf('h%c_%s', cls, v{1}), H.(cls).(v{1}).m, H.(cls).(v{1}).s}; end %#ok<AGROW>
    for side = 'RL'
        Y = C.(side).(v{1}); G(end+1,:) = {sprintf('s%c_%s', side, v{1}), mean(Y, 2), std(Y, 0, 2)}; %#ok<AGROW>
    end
    for g = 1:size(G, 1)
        cols = [cols, {[G{g,1} '_m'], [G{g,1} '_lo'], [G{g,1} '_hi']}]; %#ok<AGROW>
        D = [D, G{g,2}, G{g,2} - G{g,3}, G{g,2} + G{g,3}]; %#ok<AGROW>
    end
end
% apoio no saudavel: primeiro ponto apos 40 % com forca vertical < 2 % do peso
for cls = 'SC'
    fa = dir(fullfile(dsDir, sprintf('WBDS*walk0%cknt.txt', cls))); sp = [];
    for i = 1:numel(fa)
        k = readtable(fullfile(fa(i).folder, fa(i).name), 'FileType', 'text', 'Delimiter', '\t');
        for f = {'RGRFY', 'LGRFY'}
            y = k.(f{1}) / 9.81; j = find(pct > 40 & y < 0.02, 1);
            if ~isempty(j), sp(end+1) = pct(j); end %#ok<AGROW>
        end
    end
    H.(cls).stance = [mean(sp), std(sp)];
end
Tt = array2table(D, 'VariableNames', cols);
writetable(Tt, fullfile(figDir, 'ciclo_comparacao.csv'));
% faixas como poligonos fechados (limite superior 0->100 e inferior 100->0),
% para o pgfplots preencher sem costuras
B = table([pct; flipud(pct)], 'VariableNames', {'bx'});
for c = 2:3:numel(cols)
```

```
    name = erase(cols{c}, '_m');
    B.(name) = [Tt.([name '_hi']); flipud(Tt.([name '_lo']))];
end
writetable(B, fullfile(figDir, 'ciclo_faixas.csv'));

R = table();
for side = 'RL'
    for v = vars
        ym = mean(C.(side).(v{1}), 2); hm = H.S.(v{1}).m; hs = H.S.(v{1}).s;
        cc = corrcoef(ym, hm);
        R = [R; table(string(side), string(v{1}), sqrt(mean((ym - hm).^2)), cc(1,2), ...
            100*mean(abs(ym - hm) <= hs), 'VariableNames', {'lado','variavel','RMSE','corr','pctDentroDP'})];
      %#ok<AGROW>
    end
end
writetable(R, fullfile(figDir, 'ciclo_resumo.csv'));
fprintf('passadas: D=%d  E=%d | sujeitos saudaveis: S=%d  C=%d | massa do modelo=%.1f kg\n', ...
    nStr.R, nStr.L, H.S.hip.n, H.C.hip.n, BW/9.81);
fprintf('apoio (%% do ciclo): protese %.1f +/- %.1f | intacta %.1f +/- %.1f | saudavel lento %.1f +/- %.1f |
     confortavel %.1f +/- %.1f\n', ...
    mean(stance.R), std(stance.R), mean(stance.L), std(stance.L), H.S.stance, H.C.stance);
disp(R);
S = struct('H', H, 'C', C, 'stance', stance, 'nStrides', nStr, 'summary', R);
save(fullfile(figDir, 'ciclo_comparacao.mat'), 'S');
end
```

B.8 make_media.m

```
function make_media(tag, varargin)
%MAKE_MEDIA Figura-resumo (PNG) e video (MP4) de uma avaliacao salva em
%   results/<tag>_traj.mat.
%   Funciona em "matlab -batch": tudo e desenhado direto em pixels
%   (insertShape/insertText), sem figuras do MATLAB, porque o sistema
%   grafico trava nesta maquina.
%
%   make_media('chunk093')
%   make_media('chunk093', 'fps', 30, 'slow', 2)   % camera lenta 2x

p = inputParser;
p.addParameter('fps', 30);
p.addParameter('slow', 1);
p.parse(varargin{:}); o = p.Results;
import org.opensim.modeling.*

here = fileparts(mfilename('fullpath'));
outDir = fullfile(here, '..', 'results');
S = load(fullfile(outDir, [tag '_traj.mat'])); tr = S.tr; cfg = S.cfg; M = S.M;
ref = healthyGaitReference();
n = numel(tr.t);

% ---------- cinematica direta: pontos do corpo no plano sagital ----------
model = Model(cfg.modelFile); s = model.initSystem();
CS = model.getCoordinateSet(); BS = model.getBodySet(); CG = model.getContactGeometrySet();
bodies = {'pelvis','transfemur','tibia_pylon','foot_r','femur_l','tibia_l','talus_l','torso'};
spheres = {'r_heel','r_toe1','l_heel','l_toe1'};
P = struct();
for k = [bodies spheres {'head'}], P.(k{1}) = zeros(n, 2); end
for i = 1:n
    for j = 1:numel(cfg.allCoords), CS.get(cfg.allCoords{j}).setValue(s, tr.q(i,j), false); end
    model.realizePosition(s);
    for k = 1:numel(bodies)
        v = BS.get(bodies{k}).getPositionInGround(s); P.(bodies{k})(i,:) = [v.get(0) v.get(1)];
    end
    v = BS.get('torso').findStationLocationInGround(s, Vec3(0, 0.55, 0)); P.head(i,:) = [v.get(0) v.get(1)];
    for k = 1:numel(spheres)
        g = CG.get(spheres{k});
        v = g.getFrame().findStationLocationInGround(s, g.get_location()); P.(spheres{k})(i,:) = [v.get(0) v.
     get(1)];
    end
end
writematrix([tr.t cell2mat(cellfun(@(k) P.(k), [bodies spheres {'head'}], 'UniformOutput', false))], ...
    fullfile(outDir, [tag '_pontos.csv']));
```

```

cR = [217 84 26]; cL = [0 114 189]; cT = [70 70 70]; cG = [150 150 150];

% ================== FIGURA-RESUMO ==================
W = 1800; H = 1400; img = uint8(255 * ones(H, W, 3));
img = txt(img, [W/2 18], sprintf(['%s  |  %.1f s em pe  |  v = %.2f m/s (saudavel: 1,2)  |  RMSE = %.1f°  |  ...
    'corr = %.2f  |  assimetria de apoio = %.0f%%'], tag, M.survivalTime, M.meanSpeed, M.rmseMean, M.corrMean, M.stanceSymmetry), ...
    24, [0 0 0], 'CenterTop');

% --- sequencia de bonecos ---
xs = [P.r_heel(:,1); P.l_toe1(:,1); P.head(:,1)];
pnl = panel([60 80 W-120 330], [min(xs)-0.1 max(xs)+0.1], [-0.05 1.75], true);
img = axesBox(img, pnl, 'Sequencia a cada 0,1 s  (laranja: protese passiva D, azul: perna intacta E)', 'x (m)', '', false);
img = shape(img, 'Line', [pnl.px(pnl.xl(1)) pnl.py(0) pnl.px(pnl.xl(2)) pnl.py(0)], [0 0 0], 2);
stepK = max(1, round(0.1 / mean(diff(tr.t))));
for i = 1:stepK:n
    a = 0.25 + 0.75 * i / n;
    img = drawStick(img, P, i, pnl, fade(cR, a), fade(cL, a), fade(cT, a), 2);
end

% --- angulos articulares x referencia saudavel ---
ph = mod(cfg.initPhase + tr.t / ref.cycleTime, 1);
idx = [7 10 11 12 14 15]; sig = {ref.hip, ref.knee, ref.ankle, ref.hip, ref.knee, ref.ankle};
off = [0.5 0.5 0.5 0 0 0]; rn = {'hipR','kneeR','ankR','hipL','kneeL','ankL'};
ttl = {'Quadril D (coto)','Joelho D (protese)','Tornozelo D (protese)','Quadril E','Joelho E','Tornozelo E'};
pw = (W - 120 - 2*70) / 3; ph0 = 470; phh = 230;
for j = 1:6
    col = mod(j-1, 3); row = floor((j-1) / 3);
    r = rad2deg(interp1((0:ref.N)'/ref.N, [sig{j}; sig{j}(1)], mod(ph + off(j), 1)));
    y = rad2deg(tr.q(:, idx(j)));
    yl = [min([r; y]) max([r; y])]; yl = yl + [-0.08 0.08] * diff(yl);
    pn = panel([60 + col*(pw+70), ph0 + row*(phh+80), pw, phh], [tr.t(1) tr.t(end)], yl, false);
    img = axesBox(img, pn, sprintf('%s   RMSE %.0f°', ttl{j}, M.rmse.(rn{j})), '', 'graus', row == 1);
    c = cR; if j > 3, c = cL; end
    img = polyline(img, pn, tr.t, r, [40 40 40], 2, true);
    img = polyline(img, pn, tr.t, y, c, 3, false);
    if j == 1
        img = txt(img, [pn.rect(1)+pn.rect(3)-10 pn.rect(2)+6], '- - saudavel (ref.)', 15, [0 0 0], 'RightTop');
        img = txt(img, [pn.rect(1)+pn.rect(3)-10 pn.rect(2)+26], '-- simulado', 15, c, 'RightTop');
    end
end

% --- GRF, excitacoes, pelve ---
y0 = 1095; hh = 240;
g = tr.grf; pn = panel([60, y0, pw, hh], [tr.t(1) tr.t(end)], [0 max(g(:))*1.08 + 1], false);
img = axesBox(img, pn, 'Forca vertical do solo (N)', 't (s)', 'N', true);
img = polyline(img, pn, tr.t, g(:,1), cR, 2, false);
img = polyline(img, pn, tr.t, g(:,2), cL, 2, false);
img = txt(img, [pn.rect(1)+pn.rect(3)-10 pn.rect(2)+6], 'D (protese)', 15, cR, 'RightTop');
img = txt(img, [pn.rect(1)+pn.rect(3)-10 pn.rect(2)+26], 'E (intacta)', 15, cL, 'RightTop');

x1 = 60 + (pw+70) + 110; pn = panel([x1, y0, pw - 110, hh], [tr.t(1) tr.t(end)], [0.5 15.5], false);
E = tr.e(:,1:15)'; cm = round(255 * parula(256));
Ei = imresize(E, [hh round(pw - 110)], 'nearest'); Ei = min(256, max(1, round(Ei * 255) + 1));
x1 = round(x1); img(y0+1:y0+hh, x1+1:x1+round(pw-110), :) = reshape(uint8(cm(Ei, :)), [hh round(pw-110) 3]);
img = axesBox(img, pn, 'Excitacoes musculares (0 a 1)', 't (s)', '', true, false);
for k = 1:15
    img = txt(img, [x1 - 6, pn.py(16 - k) - 8], cfg.muscles{k}, 12, [0 0 0], 'RightTop');
end

x2 = 60 + 2*(pw+70); v = [NaN; diff(tr.q(:,4)) ./ diff(tr.t)]; v = movmean(v, 11, 'omitnan');
pn = panel([x2, y0, pw, hh], [tr.t(1) tr.t(end)], [0 1.6], false);
img = axesBox(img, pn, 'Pelve: altura (m) e velocidade v_x (m/s)', 't (s)', '', true);
img = shape(img, 'Line', [pn.px(tr.t(1)) pn.py(1.2) pn.px(tr.t(end)) pn.py(1.2)], cG, 1);
img = polyline(img, pn, tr.t, tr.q(:,5), cT, 3, false);
img = polyline(img, pn, tr.t, v, [119 172 48], 2, false);
img = txt(img, [pn.rect(1)+pn.rect(3)-10 pn.rect(2)+6], 'altura', 15, cT, 'RightTop');
img = txt(img, [pn.rect(1)+pn.rect(3)-10 pn.rect(2)+26], 'velocidade (cinza: 1,2 m/s)', 15, [119 172 48], 'RightTop');

figFile = fullfile(outDir, [tag '_resultado.png']); imwrite(img, figFile);
```

```
fprintf('figura: %s\n', figFile);

% ================== VIDEO ==================
VW = 1280; VH = 720; sc = 330;               % pixels por metro
vidFile = fullfile(outDir, [tag '_marcha.mp4']);
vw = VideoWriter(vidFile, 'MPEG-4'); vw.FrameRate = o.fps; vw.Quality = 90; open(vw);
tq = tr.t(1):1/(o.fps*o.slow):tr.t(end);
xc = movmean(P.pelvis(:,1), 25);              % camera suave
for tk = tq
    [~, i] = min(abs(tr.t - tk));
    pv = panel([0 0 VW VH], xc(i) + [-VW/2 VW/2]/sc, [-0.25, VH/sc - 0.25], true);
    fr = uint8(255 * ones(VH, VW, 3));
    gy = pv.py(0);
    fr(round(gy):end, :, :) = 235;
    for gx = floor(pv.xl(1)*4)/4 : 0.25 : pv.xl(2)
        fr = shape(fr, 'Line', [pv.px(gx) gy pv.px(gx) gy+14], cG, 2);
    end
    fr = shape(fr, 'Line', [0 gy VW gy], [0 0 0], 3);
    fr = drawStick(fr, P, i, pv, cR, cL, cT, 7);
    sc_g = 0.35 / (9.81 * 70);                % 1 peso corporal = 0,35 m de seta
    feet = {[P.r_heel(i,1) P.r_toe1(i,1)], [P.l_heel(i,1) P.l_toe1(i,1)]}; cc = {cR, cL};
    for k = 1:2
        if tr.grf(i,k) > 20
            x = mean(feet{k}); L = tr.grf(i,k) * sc_g;
            fr = shape(fr, 'Line', [pv.px(x) gy pv.px(x) pv.py(L)], cc{k}, 4);
            fr = shape(fr, 'FilledPolygon', [pv.px(x)-9 pv.py(L)+2 pv.px(x)+9 pv.py(L)+2 pv.px(x) pv.py(L)
     -14], cc{k}, 1);
        end
    end
    fr = txt(fr, [20 15], sprintf('t = %.2f s     v media = %.2f m/s     %s', tr.t(i), M.meanSpeed, tag), 26,
      [0 0 0], 'LeftTop');
    fr = txt(fr, [20 55], 'laranja: protese passiva (joelho mola+amortecedor, pe ESAR)   azul: perna intacta
       setas: forca do solo', 18, cT, 'LeftTop');
    if o.slow ~= 1, fr = txt(fr, [VW-20 15], sprintf('camera lenta %gx', o.slow), 22, cT, 'RightTop'); end
    writeVideo(vw, fr);
end
close(vw);
fprintf('video: %s (%d quadros)\n', vidFile, numel(tq));
end

% ================== auxiliares de desenho ==================
function pn = panel(rect, xl, yl, equalAxes)
% rect = [x y w h] em pixels; xl/yl = faixa de dados
if equalAxes      % mesma escala em x e y (para os bonecos)
    sc = min(rect(3) / diff(xl), rect(4) / diff(yl));
    xl = mean(xl) + [-0.5 0.5] * rect(3) / sc;
    yl = yl(1) + [0 1] * rect(4) / sc;
end
pn.rect = rect; pn.xl = xl; pn.yl = yl;
pn.px = @(x) rect(1) + (x - xl(1)) / diff(xl) * rect(3);
pn.py = @(y) rect(2) + rect(4) - (y - yl(1)) / diff(yl) * rect(4);
end

function img = shape(img, kind, pos, c, w)
if any(~isfinite(pos(:))), return; end
if strcmp(kind, 'Line')
    img = insertShape(img, 'Line', pos, 'Color', c, 'LineWidth', w, 'SmoothEdges', true);
elseif strcmp(kind, 'FilledPolygon') || strcmp(kind, 'FilledCircle')
    img = insertShape(img, kind, pos, 'Color', c, 'Opacity', w, 'SmoothEdges', true);
else
    img = insertShape(img, kind, pos, 'Color', c, 'LineWidth', w, 'SmoothEdges', true);
end
end

function img = txt(img, pos, str, fs, c, anchor)
img = insertText(img, pos, str, 'FontSize', fs, 'TextColor', c, 'BoxOpacity', 0, 'AnchorPoint', anchor);
end

function c = fade(c, a)
c = round(c * a + 255 * (1 - a));
end

function img = polyline(img, pn, x, y, c, w, dashed)
ok = isfinite(x) & isfinite(y); x = x(ok); y = y(ok);
X = pn.px(x(:)); Y = pn.py(y(:));
```

```
if dashed
    seg = 4; % pontos por traco
    for k = 1:round(1.6*seg):numel(X)-1
        e = min(numel(X), k + seg);
        pts = [X(k:e) Y(k:e)]'; img = shape(img, 'Line', pts(:)', c, w);
    end
else
    pts = [X Y]'; img = shape(img, 'Line', pts(:)', c, w);
end
end

function img = axesBox(img, pn, ttl, xlab, ylab, showX, showY)
if nargin < 7, showY = true; end
r = pn.rect;
img = insertShape(img, 'Rectangle', r, 'Color', [120 120 120], 'LineWidth', 1);
img = txt(img, [r(1) r(2) - 30], ttl, 19, [0 0 0], 'LeftTop');
yt = niceTicks(pn.yl); if ~showY, yt = []; end
for v = yt
    y = pn.py(v);
    img = shape(img, 'Line', [r(1) y r(1)+r(3) y], [225 225 225], 1);
    img = txt(img, [r(1) - 6, y - 9], num2str(v), 13, [60 60 60], 'RightTop');
end
xt = niceTicks(pn.xl);
for v = xt
    x = pn.px(v);
    img = shape(img, 'Line', [x r(2)+r(4) x r(2)+r(4)+5], [120 120 120], 1);
    if showX || isempty(ylab), img = txt(img, [x, r(2)+r(4)+7], num2str(v), 13, [60 60 60], 'CenterTop'); end
end
if ~isempty(xlab) && showX, img = txt(img, [r(1)+r(3), r(2)+r(4)+26], xlab, 14, [60 60 60], 'RightTop'); end
if ~isempty(ylab), img = txt(img, [r(1) - 50, r(2) - 4], ylab, 13, [60 60 60], 'LeftTop'); end
end

function t = niceTicks(lim)
d = diff(lim) / 5; m = 10^floor(log10(d)); d = m * [1 2 5 10]; d = d(find(d >= diff(lim)/6, 1));
t = ceil(lim(1)/d)*d : d : lim(2);
t = round(t, 6);
end

function img = drawStick(img, P, i, pn, cR, cL, cT, lw)
pt = @(nm) [pn.px(P.(nm)(i,1)) pn.py(P.(nm)(i,2))];
line = @(img, names, c, w) shape(img, 'Line', reshape(cell2mat(cellfun(pt, names, 'UniformOutput', false))', 1, []), c, w);
poly = @(names) reshape(cell2mat(cellfun(pt, names, 'UniformOutput', false))', 1, []);
% perna intacta (atras), tronco, protese (frente, lado direito visto de lado)
img = line(img, {'pelvis','femur_l','tibia_l','talus_l'}, cL, lw);
img = shape(img, 'FilledPolygon', poly({'talus_l','l_heel','l_toe1'}), cL, 0.6);
img = line(img, {'pelvis','torso','head'}, cT, lw + 2);
img = line(img, {'pelvis','transfemur','tibia_pylon','foot_r'}, cR, lw);
img = shape(img, 'FilledPolygon', poly({'foot_r','r_heel','r_toe1'}), cR, 0.6);
if lw >= 5
    h = pt('head'); rh = 0.11 * (pn.rect(3) / diff(pn.xl));
    img = shape(img, 'FilledCircle', [h(1) h(2) - rh*0.8 rh], [255 255 255], 1);
    img = shape(img, 'Circle', [h(1) h(2) - rh*0.8 rh], cT, 3);
    for nm = {'femur_l','tibia_l','talus_l','transfemur','tibia_pylon','foot_r','pelvis'}
        q = pt(nm{1}); img = shape(img, 'FilledCircle', [q lw*0.9], [255 255 255], 1);
        img = shape(img, 'Circle', [q lw*0.9], cT, 2);
    end
end
end
```

B.9 render_opensim.m

```
function render_opensim(tag, varargin)
%RENDER_OPENSIM Video da marcha no visualizador 3D do OpenSim (Simbody
%   visualizer, com os ossos, a protese e os musculos do modelo). Reproduz a
%   trajetoria salva em results/<tag>_traj.mat, captura cada quadro da
%   janela do visualizador e grava results/<tag>_opensim.mp4.
%   A janela do visualizador precisa ficar visivel (nao minimizada, sem
%   outras janelas por cima) durante a gravacao.
%
%   render_opensim('cmp_chunk129_lr1e-4_ckpt_p00')
%   render_opensim(tag, 'fps', 30, 'maxTime', 10, 'testFrame', true)
```

```
11
12  p = inputParser;
13  p.addParameter('fps', 30);
14  p.addParameter('maxTime', inf);
15  p.addParameter('testFrame', false);      % so 1 quadro (para conferir enquadramento)
16  p.addParameter('camDist', 2.6);          % distancia da camera (m)
17  p.parse(varargin{:}); o = p.Results;
18  import org.opensim.modeling.*
19
20  here = fileparts(mfilename('fullpath'));
21  outDir = fullfile(here, '..', 'results');
22  S = load(fullfile(outDir, [tag '_traj.mat'])); tr = S.tr; cfg = S.cfg; M = S.M;
23
24  ModelVisualizer.addDirToGeometrySearchPaths(fullfile(fileparts(cfg.modelFile), 'Geometry'));
25  ModelVisualizer.addDirToGeometrySearchPaths('C:\OpenSim 4.4\Geometry');   % ossos padrao (.vtp)
26  model = Model(cfg.modelFile);
27  model.setUseVisualizer(true);
28  s = model.initSystem();
29  sviz = model.updVisualizer().updSimbodyVisualizer();
30  sviz.setShowSimTime(true);
31  sviz.setShowFrameRate(false);
32  CS = model.getCoordinateSet();
33  capt = fullfile(here, 'capture_window.ps1');
34  frDir = fullfile(tempdir, ['osimframes_' tag]); if ~exist(frDir, 'dir'), mkdir(frDir); end
35
36  tEnd = min(tr.t(end), o.maxTime);
37  tq = tr.t(1):1/o.fps:tEnd;
38  if o.testFrame, tq = tr.t(round(end/2)); end
39  xc = movmean(tr.q(:,4), 25);               % camera acompanha a pelve (suave)
40  for k = 1:numel(tq)
41      [~, i] = min(abs(tr.t - tq(k)));
42      s.setTime(tr.t(i));
43      for j = 1:numel(cfg.allCoords), CS.get(cfg.allCoords{j}).setValue(s, tr.q(i,j), false); end
44      model.realizePosition(s);
45      % vista lateral (plano sagital), olhando o lado direito (protese)
46      sviz.setCameraTransform(Transform(Vec3(xc(i), 0.8, o.camDist)));
47      model.getVisualizer().show(s);
48      if k == 1, pause(1.5); else, pause(0.03); end
49      f = fullfile(frDir, sprintf('f%05d.png', k));
50      front = ''; if k == 1, front = ' -front'; end
51      for tent = 1:4   % confere se o quadro mostra a cena (ceu azul no alto); se outra janela cobriu, captura
         de novo
52          [st, msg] = system(sprintf('powershell -NoProfile -ExecutionPolicy Bypass -File "%s" -out "%s"%s',
         capt, f, front));
53          if st ~= 0, error('captura falhou: %s', msg); end
54          I = double(imread(f)); sky = I(60:120, round(end*0.7):end-10, :);
55          if mean(sky(:,:,3), 'all') > mean(sky(:,:,1), 'all') + 40, break; end
56          fprintf('quadro %d coberto por outra janela, capturando de novo\n', k);
57          front = ' -front'; model.getVisualizer().show(s); pause(0.3);
58      end
59      if mod(k, 60) == 0, fprintf('quadro %d/%d\n', k, numel(tq)); end
60  end
61
62  if o.testFrame
63      copyfile(f, fullfile(outDir, [tag '_opensim_teste.png']));
64      fprintf('teste: %s\n', fullfile(outDir, [tag '_opensim_teste.png']));
65      return
66  end
67  vidFile = fullfile(outDir, [tag '_opensim.mp4']);
68  vw = VideoWriter(vidFile, 'MPEG-4'); vw.FrameRate = o.fps; vw.Quality = 90; open(vw);
69  I0 = imread(fullfile(frDir, 'f00001.png')); sz = size(I0); sz = 2*floor(sz(1:2)/2);
70  for k = 1:numel(tq)
71      I = imread(fullfile(frDir, sprintf('f%05d.png', k)));
72      I = I(1:sz(1), 1:sz(2), :);
73      I = insertText(I, [10 10], sprintf('v media = %.2f m/s  |  protese passiva: perna direita  |  %s', M.
         meanSpeed, tag), ...
74          'FontSize', 16, 'BoxOpacity', 0.4);
75      writeVideo(vw, I);
76  end
77  close(vw);
78  fprintf('video: %s (%d quadros)\n', vidFile, numel(tq));
79  end
```

B.10 capture_window.ps1

```
# Captura a area cliente da janela do visualizador Simbody (OpenSim) para um PNG.
# Uso: powershell -File capture_window.ps1 -out C:\caminho\frame.png [-front]
param([string]$out, [switch]$front, [string]$proc = "simbody-visualizer")
Add-Type -AssemblyName System.Drawing
Add-Type @"
using System; using System.Runtime.InteropServices;
public class W {
  [DllImport("user32.dll")] public static extern bool SetProcessDPIAware();
  [DllImport("user32.dll")] public static extern bool GetClientRect(IntPtr h, out R r);
  [DllImport("user32.dll")] public static extern bool ClientToScreen(IntPtr h, ref P p);
  [DllImport("user32.dll")] public static extern bool SetForegroundWindow(IntPtr h);
  [DllImport("user32.dll")] public static extern bool ShowWindow(IntPtr h, int c);
  [DllImport("user32.dll")] public static extern bool SetWindowPos(IntPtr h, IntPtr a, int x, int y, int cx,
     int cy, uint f);
  public struct R { public int L, T, Ri, B; }
  public struct P { public int X, Y; }
}
"@
[void][W]::SetProcessDPIAware()
$p = Get-Process $proc -ErrorAction Stop | Where-Object { $_.MainWindowHandle -ne 0 } | Select-Object -First
     1
$h = $p.MainWindowHandle
# -front: restaura, traz para frente e deixa SEMPRE POR CIMA (HWND_TOPMOST) durante a gravacao
if ($front) { [void][W]::ShowWindow($h, 9); [void][W]::SetWindowPos($h, [IntPtr](-1), 0, 0, 0, 0, 3); [void][
     W]::SetForegroundWindow($h); Start-Sleep -Milliseconds 300 }
$r = New-Object W+R; [void][W]::GetClientRect($h, [ref]$r)
$pt = New-Object W+P; [void][W]::ClientToScreen($h, [ref]$pt)
$w = $r.Ri - $r.L; $hh = $r.B - $r.T
$bmp = New-Object System.Drawing.Bitmap $w, $hh
$g = [System.Drawing.Graphics]::FromImage($bmp)
$g.CopyFromScreen($pt.X, $pt.Y, 0, 0, $bmp.Size)
$bmp.Save($out, [System.Drawing.Imaging.ImageFormat]::Png)
$g.Dispose(); $bmp.Dispose()
"$w x $hh"
```

B.11 bench_gpu.m

```
% Mede o custo do APRENDIZ do SAC (gradientes das redes) em CPU vs GPU,
% isolado da simulacao: usa um ambiente falso instantaneo com as mesmas
% dimensoes do ProstheticGaitEnv (obs 63, acao 15) e o mesmo agente.
obsInfo = rlNumericSpec([63 1]);
actInfo = rlNumericSpec([15 1], 'LowerLimit', -1, 'UpperLimit', 1);
env = rlFunctionEnv(obsInfo, actInfo, @(a, lg) deal(randn(63,1), rand, false, lg), @() deal(randn(63,1), [])
     ;
nSteps = 400; warm = 300;
for dev = ["cpu" "gpu"]
    agent = rlSACAgent(obsInfo, actInfo, rlAgentInitializationOptions('NumHiddenUnit', 256));
    ao = agent.AgentOptions;
    ao.MiniBatchSize = 256; ao.NumWarmStartSteps = warm; ao.LearningFrequency = 1;
    agent.AgentOptions = ao;
    actor = getActor(agent); actor.UseDevice = dev; agent = setActor(agent, actor);
    critics = getCritic(agent); for i = 1:numel(critics), critics(i).UseDevice = dev; end
    agent = setCritic(agent, critics);
    topts = rlTrainingOptions('MaxEpisodes', 1, 'MaxStepsPerEpisode', warm, 'Plots', 'none', 'Verbose', false
     , 'StopTrainingCriteria', 'none');
    train(agent, env, topts);                          % aquece (preenche buffer, compila)
    topts.MaxStepsPerEpisode = nSteps;
    tic; train(agent, env, topts); el = toc;
    fprintf('%s: %.2f ms por passo com 1 gradiente/passo (%d passos em %.1f s)\n', dev, 1000*el/nSteps,
     nSteps, el);
end
```

B.12 test_deriv.m

```
% Compara integradores do MATLAB usando o OpenSim apenas para ydot = f(t,y)
import org.opensim.modeling.*
cfg = gaitConfig('randomInitPhase', false);
```

```
env = ProstheticGaitEnv(cfg); reset(env);
m = env.model; s = org.opensim.modeling.State(env.state);
y0 = s.getY().getAsMat();
fprintf('nY=%d nQ=%d nU=%d nZ=%d\n', numel(y0), s.getNQ(), s.getNU(), s.getNZ());
f = @(t,y) deriv(m, s, t, y);
tic; for k=1:200, d = f(0, y0); end; fprintf('deriv: %.2f ms\n', toc/200*1000);
global NF
for solver = {@ode45, @ode23, @ode15s, @ode23t, @ode23tb}
  NF = 0; tic;
  opts = odeset('RelTol',1e-3,'AbsTol',1e-4);
  try
    [T,Y] = solver{1}(f, [0 0.1], y0, opts);
    fprintf('%s: 0.1 s em %.2f s, %d avaliacoes, %d passos, ty fim=%.3f\n', func2str(solver{1}), toc, NF,
     numel(T), Y(end,5));
  catch e, fprintf('%s falhou: %s\n', func2str(solver{1}), e.message); end
end
function d = deriv(m, s, t, y)
global NF; NF = NF + 1;
s.setTime(t); s.setY(org.opensim.modeling.Vector.createFromMat(y));
m.realizeAcceleration(s);
d = s.getYDot().getAsMat();
end
```

B.13 test_baseline.m

```
% Testa o ambiente com acao zero (so controlador-base) com e sem assistencia
for assist = [1 0.5 0]
    cfg = gaitConfig('randomInitPhase', false, 'initPhase', 0.0, 'assistStart', assist);
    env = ProstheticGaitEnv(cfg);
    env.record = true;
    obs = reset(env);
    R = 0; tic;
    for k = 1:cfg.maxSteps
        [obs, r, done, info] = step(env, zeros(15,1));
        R = R + r;
        if done, break; end
    end
    el = toc;
    tr = env.traj;
    vx = (tr.q(end,4) - 0) / tr.t(end);
    fprintf('assist=%.1f: %d passos (%.2f s sim) em %.1f s | R=%.1f | vx med=%.2f | ty fim=%.2f | tilt fim
     =%.2f\n', ...
        assist, k, tr.t(end), el, R, vx, tr.q(end,5), tr.q(end,1));
    fprintf('   pose=%.2f speed=%.2f upright=%.2f effort=%.3f | GRF max r/l = %.0f/%.0f N\n', ...
        info.pose, info.speed, info.upright, info.effort, max(tr.grf(:,1)), max(tr.grf(:,2)));
    fprintf('   faixa (graus): hipR [%.0f %.0f] kneeR [%.0f %.0f] ankR [%.0f %.0f] hipL [%.0f %.0f] kneeL
     [%.0f %.0f] ankL [%.0f %.0f]\n', ...
        rad2deg([min(tr.q(:,7)) max(tr.q(:,7)) min(tr.q(:,10)) max(tr.q(:,10)) min(tr.q(:,11)) max(tr.q(:,11)
     ) ...
                 min(tr.q(:,12)) max(tr.q(:,12)) min(tr.q(:,14)) max(tr.q(:,14)) min(tr.q(:,15)) max(tr.q
     (:,15))]));
end
```

B.14 sweep.m

```
function sweep(varargin)
% sweep('param',{v1,v2,...}, extras...) roda o controlador-base (acao zero, sem assistencia)
base = {'randomInitPhase', false, 'assistStart', 0};
name = varargin{1}; vals = varargin{2}; extra = varargin(3:end);
if ~iscell(vals), vals = num2cell(vals); end
for iv = 1:numel(vals)
  v = vals{iv};
  for ph = [0 0.5]
    cfg = gaitConfig(base{:}, 'initPhase', ph, extra{:}, name, v);
    env = ProstheticGaitEnv(cfg); env.record = true; reset(env);
    P = 0;
    for k = 1:cfg.maxSteps
        [~, ~, done, info] = step(env, zeros(15,1)); P = P + info.pose;
        if done, break; end
    end
```

```
    tr = env.traj; qe = rad2deg(tr.q(end,:));
    fprintf('%s=%s ph=%.1f: %3d passos | pose=%.2f | vx=%.2f | tilt=%.2f ty=%.2f | fim: hipR %.0f kneeR %.0f ankR %.0f hipL %.0f kneeL %.0f ankL %.0f\n', ...
        name, mat2str(v), ph, k, P/k, tr.q(end,4)/tr.t(end), tr.q(end,1), tr.q(end,5), qe([7 10 11 12 14 15]));
  end
end
end
```